

# TERRAFORM COMPLETE GUIDE

BEGINNER → PRO

Infrastructure as Code using Terraform with AWS

AUTHOR: ENGINEER\_CLOUDOPS

[infracraft.dev](https://infracraft.dev) | [@engineer\\_cloudops](https://twitter.com/engineer_cloudops)

---

# TABLE OF CONTENTS

## PART I - FOUNDATIONS

- Chapter 1 — Introduction to Infrastructure as Code
- Chapter 2 — Terraform Architecture
- Chapter 3 — Installing Terraform
- Chapter 4 — Terraform CLI
- Chapter 5 — Terraform Language Basics (HCL)

## PART II - CORE LANGUAGE

- Chapter 6 — Variables
- Chapter 7 — Outputs
- Chapter 8 — Locals
- Chapter 9 — Providers
- Chapter 10 — Resources
- Chapter 11 — Data Sources
- Chapter 12 — Expressions
- Chapter 13 — Functions
- Chapter 14 — Meta-Arguments
- Chapter 15 — Dynamic Blocks

## PART III - SCALING TERRAFORM

- Chapter 16 — Modules
- Chapter 17 — Remote State
- Chapter 18 — State File Deep Dive
- Chapter 19 — Terraform Workspaces
- Chapter 20 — Provisioners

## PART IV - AWS IN PRODUCTION

- Chapter 21 — AWS Infrastructure Examples
- Chapter 22 — End-to-End Production Project
- Chapter 23 — CI/CD Integration
- Chapter 24 — Terraform Best Practices
- Chapter 25 — Production Troubleshooting

## PART V - CAREER & MASTERY

- Chapter 26 — Terraform Interview Questions
- Chapter 27 — Real Production Scenarios
- Chapter 28 — Terraform Cheat Sheets
- Chapter 29 — Hands-On Labs
- Chapter 30 — Final Notes - Career Roadmap

---

## PART I - FOUNDATIONS

# Chapter 1: Introduction to Infrastructure as Code

Imagine you join a new company on a Monday morning. Your manager asks you to set up a production environment: a VPC, three subnets, an EC2 fleet, a load balancer, and an RDS database. If you did this by clicking through the AWS Console, it would take hours, it would be undocumented, and no one else could reproduce it. Infrastructure as Code (IaC) solves exactly this problem: it lets you describe your entire infrastructure in text files, version them in Git, and recreate them anywhere with a single command.

## What is Infrastructure as Code (IaC)

IaC is the practice of managing and provisioning infrastructure (servers, networks, load balancers, databases, DNS records) through machine-readable configuration files, instead of manual configuration or interactive tools. The configuration file becomes the single source of truth for what your infrastructure should look like.

## Traditional Infrastructure vs IaC

| Aspect             | Traditional (Manual)                 | Infrastructure as Code         |
|--------------------|--------------------------------------|--------------------------------|
| Provisioning       | Manual clicks in AWS Console         | Declarative code (.tf files)   |
| Repeatability      | Different every time (human error)   | Identical every time           |
| Documentation      | Tribal knowledge / wiki (goes stale) | Code is the documentation      |
| Version Control    | Not possible                         | Full Git history, PR review    |
| Rollback           | Manual, risky, slow                  | git revert + terraform apply   |
| Environment Parity | Dev/QA/Prod drift over time          | Same code, different variables |
| Audit Trail        | CloudTrail only, no intent captured  | Code diff shows exact intent   |

## Benefits of IaC

- **Speed:** Spin up a full environment in minutes instead of days.
- **Consistency:** Dev, QA, and Production are built from the same templates.
- **Version Control:** Every change goes through Git, so you get history, blame, and review.
- **Disaster Recovery:** Lost your environment? Run terraform apply again.
- **Cost Control:** Tear down non-production environments overnight with terraform destroy.
- **Collaboration:** Multiple engineers can review infrastructure changes like application code.

## Why Terraform (and not just AWS CloudFormation or Ansible)

---

AWS CloudFormation only manages AWS. Ansible is push-based configuration management, better suited for configuring servers after they exist. Terraform, built by HashiCorp, is **cloud-agnostic**, works with over 3,000 providers (AWS, Azure, GCP, Kubernetes, Datadog, GitHub, and more), uses a human-readable declarative language called HCL, and maintains a **state file** that tracks exactly what it created. This makes Terraform the industry standard for multi-cloud and hybrid infrastructure automation - and the single most in-demand IaC skill in DevOps job postings today.

**NOTE:** In real BFSI/enterprise environments (banking, fintech), Terraform is almost always paired with a governance layer - custom internal providers, Sentinel/OPA policy checks, and a centralized state backend - to enforce compliance before any apply reaches production.

**Fig 1.1 - How Terraform Fits Into Your Workflow**

```
DEVELOPER TERRAFORM CORE CLOUD PROVIDER
-----
main.tf ---plan--> [ Parse HCL ] AWS
variables.tf [ Build Graph ] ---API calls--> Azure
outputs.tf [ Diff State ] GCP
[ Execute Plan ] <---responses---
|
terraform.tfstate
```

---

## Chapter Summary

- IaC replaces manual, error-prone provisioning with version-controlled configuration files.
- Terraform is cloud-agnostic and uses a state file to track real infrastructure.
- IaC gives you speed, consistency, auditability, and safe disaster recovery.

## Best Practices

- Always store your Terraform code in Git, never on a local laptop only.
- Treat infrastructure code with the same review rigor as application code (PRs, approvals).
- Start every new project by deciding your state backend before writing resources.

## Common Mistakes

- Manually editing resources in the AWS Console after they are managed by Terraform (causes drift).
- Treating Terraform code as a one-time script instead of a living, reviewed codebase.
- Not distinguishing between configuration management (Ansible) and provisioning (Terraform).

## Interview Questions

**Q1. What is Infrastructure as Code and why does it matter?**

---

IaC is managing infrastructure through machine-readable definition files rather than manual processes. It matters because it brings repeatability, version control, peer review, and fast disaster recovery to infrastructure management, the same benefits DevOps brought to application code.

### **Q2. How is Terraform different from Ansible?**

Terraform is a declarative provisioning tool that tracks state and is used to create infrastructure (VPCs, EC2, RDS). Ansible is primarily a configuration management tool, push-based, and used to configure software on servers that already exist. Many teams use both: Terraform to provision, Ansible to configure.

### **Q3. Is Terraform only for AWS?**

No. Terraform is cloud-agnostic and supports over 3,000 providers including AWS, Azure, GCP, Kubernetes, and many SaaS platforms, all using the same HCL syntax.

---

# Chapter 2: Terraform Architecture

Before writing a single line of HCL, you need a mental model of what actually happens when you run terraform apply. Terraform's architecture has four building blocks: Core, Providers, State, and the Execution Workflow.

## Terraform Core

Terraform Core is the engine, written in Go, distributed as a single binary. It is responsible for reading your .tf files, building an internal dependency graph, comparing desired state against the state file, and calling the correct provider plugins in the correct order.

## Providers and Plugins

A Provider is a plugin that knows how to talk to a specific API - the AWS provider knows how to call the AWS API, the Kubernetes provider knows how to call the Kubernetes API. Providers are downloaded into the .terraform directory the first time you run terraform init. Each provider exposes a set of Resources (things you create) and Data Sources (things you read).

## State

The state file (terraform.tfstate) is a JSON file that maps your configuration to real-world resource IDs. It is the single most important artifact in a Terraform project - without it, Terraform has no idea which AWS resources belong to it. State is covered in full detail in Chapters 17 and 18.

## Terraform Workflow: Write -> Plan -> Apply

Fig 2.1 - The Core Terraform Workflow

```
WRITE PLAN APPLY
-----
Author .tf --> terraform plan --> terraform apply
files (dry run, diff) (creates real resources)
| |
Compares: Calls Provider API
Config vs State Updates State File
```

## Execution Process, Step by Step

- **terraform init** - downloads providers, configures backend.
- **terraform validate** - checks syntax correctness.
- **terraform plan** - builds a dependency graph, compares state vs config, prints an execution plan.
- **terraform apply** - executes the plan, calling provider APIs in dependency order, updates state.
- **terraform destroy** - reverse dependency order teardown of all tracked resources.

---

**TIP:** Terraform builds a Directed Acyclic Graph (DAG) of all resources based on references between them. Resources with no dependency relationship are created in parallel, which is why Terraform is often faster than sequential shell scripts.

---

## Chapter Summary

- Terraform Core reads config, builds a dependency graph, and drives Providers.
- Providers are plugins that translate HCL into real API calls (AWS, Azure, GCP, etc).
- State is the source of truth mapping your code to real resource IDs.

## Best Practices

- Always run terraform plan and review it carefully before terraform apply.
- Pin provider versions to avoid unexpected breaking changes.
- Understand the DAG - order your code so dependencies are explicit via references, not assumptions.

## Common Mistakes

- Assuming terraform apply always runs resources top-to-bottom in file order (it uses the DAG, not file order).
- Not running terraform init after adding a new provider or module.
- Ignoring the plan output and blindly approving apply in production.

## Interview Questions

### Q1. What are the main components of Terraform's architecture?

Terraform Core (the engine), Providers (plugins for each API), the State file (source of truth), and the CLI workflow of init, plan, and apply that ties them together.

### Q2. How does Terraform decide the order in which resources are created?

It parses references between resources to build a Directed Acyclic Graph (DAG). Resources are created in dependency order, and independent resources are created in parallel.

### Q3. What happens during terraform init?

Terraform downloads the required provider plugins into .terraform/, initializes the configured backend, and downloads any referenced modules.

---

# Chapter 3: Installing Terraform

Terraform ships as a single statically-linked binary - there is no runtime dependency, no JVM, no Python interpreter needed. This chapter walks through installation on every major OS plus editor setup.

## Windows

`windows-install.ps1`

```
# Using Chocolatey (recommended)
choco install terraform

# Or manually:
# 1. Download the .zip from https://developer.hashicorp.com/terraform/downloads
# 2. Extract terraform.exe to C:\terraform
# 3. Add C:\terraform to your PATH environment variable
```

## Linux (Ubuntu / Debian / RHEL)

`linux-install.sh`

```
# Ubuntu/Debian - HashiCorp official repo
wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform -y

# RHEL / CentOS / Amazon Linux
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
sudo yum install -y terraform
```

## macOS

`macos-install.sh`

```
# Using Homebrew (recommended)
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

## Verify Installation

`verify.sh`

```
terraform -version
# Terraform v1.9.x
# on linux_amd64

terraform -help
```

## VS Code Setup

- Install the official **HashiCorp Terraform** extension from the VS Code marketplace.
- It provides syntax highlighting, auto-formatting on save, inline validation, and IntelliSense for resource arguments.
- Enable 'Format on Save' and map it to `terraform fmt` for consistent style across your team.

**TIP:** On shared build servers (Jenkins agents), pin an exact Terraform version using a version manager like `tfenv`, so every pipeline run uses the same binary regardless of what's installed on the host.

## Chapter Summary

- Terraform installs as a single binary with no runtime dependencies.
- Package managers (`choco`, `apt`, `yum`, `brew`) are the fastest, most maintainable install path.
- VS Code + HashiCorp extension gives you formatting and validation while you type.

## Best Practices

- Use a version manager (`tfenv`) to pin Terraform versions per project.
- Keep CI/CD build agents and local dev environments on the same Terraform version.

## Common Mistakes

- Installing Terraform manually and forgetting to add it to `PATH`.
- Running different Terraform versions locally vs in CI, causing state format drift.

## Interview Questions

### Q1. Does Terraform require any runtime like Java or Python?

No. Terraform is a single compiled Go binary with no external runtime dependency.

### Q2. How do you manage multiple Terraform versions across projects?

Using a version manager like `tfenv`, or pinning the `required_version` block in `terraform {}` configuration so CI fails fast if the wrong version is used.

---

# Chapter 4: Terraform CLI - Every Command Explained

The CLI is how you drive Terraform day to day. This chapter explains every important command with syntax and a real example so you never have to guess what a flag does again.

## terraform init

### terraform init [options]

Initializes a working directory: downloads providers, sets up the backend, downloads modules. Run this first, every time, and again whenever you add a provider/module/backend change.

```
terraform init -upgrade # re-download providers to their latest allowed version
```

## terraform validate

### terraform validate

Checks whether the configuration is syntactically valid and internally consistent, without touching any real infrastructure or state.

```
terraform validate
```

## terraform plan

### terraform plan [-out=planfile]

Creates an execution plan, showing what Terraform will create, change, or destroy. Nothing is applied yet.

```
terraform plan -var-file=prod.tfvars -out=tfplan
```

## terraform apply

### terraform apply [planfile]

Executes the actions in a plan (or generates a fresh plan and asks for confirmation) and updates the state file.

```
terraform apply tfplan  
terraform apply -auto-approve # skips the yes/no prompt (use carefully!)
```

## terraform destroy

### terraform destroy

Destroys every resource tracked in the state file, in reverse dependency order.

---

```
terraform destroy -target=aws_instance.web # destroy a single resource
```

## terraform fmt

**terraform fmt** [-recursive]

Rewrites .tf files into canonical formatting (indentation, alignment). Run before every commit.

```
terraform fmt -recursive
```

## terraform show

**terraform show** [planfile|statefile]

Prints a human-readable summary of the current state or a saved plan file.

```
terraform show -json terraform.tfstate | jq .
```

## terraform output

**terraform output** [name]

Prints output values from the state file. Useful for scripting (e.g., feeding a VPC ID to another pipeline stage).

```
terraform output -raw vpc_id
```

## terraform import

**terraform import**

Brings an existing, manually-created cloud resource under Terraform management by writing it into state.

```
terraform import aws_s3_bucket.logs my-existing-log-bucket
```

## terraform taint

**terraform taint** (legacy; use -replace in modern Terraform)

Marks a resource for forced recreation on the next apply, without changing its configuration.

```
terraform apply -replace="aws_instance.web" # modern equivalent
```

## terraform state

**terraform state**

A family of commands for direct state manipulation: list, show, mv, rm, pull, push.

---

```
terraform state list
terraform state show aws_instance.web
terraform state mv aws_instance.a aws_instance.b
```

## terraform workspace

### terraform workspace

Manages multiple named state files within a single configuration (new, list, select, delete).

```
terraform workspace new staging
terraform workspace select production
```

## terraform graph

### terraform graph | dot -Tsvg > graph.svg

Outputs the dependency graph in DOT format, useful for visualizing complex module relationships.

```
terraform graph -type=plan | dot -Tpng > plan_graph.png
```

## terraform refresh

### terraform apply -refresh-only (modern equivalent of terraform refresh)

Reconciles the state file with real infrastructure without changing any resources - detects drift.

```
terraform apply -refresh-only -auto-approve
```

**WARNING:** terraform apply -auto-approve should never be used against a production workspace in an interactive terminal session. Reserve -auto-approve for CI/CD pipelines where the plan has already been reviewed as a required approval gate.

---

## Chapter Summary

- init, validate, plan, apply, destroy form the core day-to-day lifecycle.
- fmt, show, output, state, and graph are inspection and maintenance commands.
- import and state mv/rm let you manage state without destroying real resources.

## Best Practices

- Always save plans with -out and apply that exact planfile in CI/CD, so what you reviewed is what runs.
- Use terraform state list regularly to sanity check what Terraform actually manages.
- Prefer terraform apply -replace over the deprecated terraform taint.

---

## Common Mistakes

- Running terraform destroy without -target and wiping out shared infrastructure by mistake.
- Forgetting that terraform fmt does not validate logic, only formatting.
- Using terraform import without first writing a matching resource block (import needs both).

## Interview Questions

### Q1. What is the difference between terraform plan and terraform apply?

plan is a dry run that shows proposed changes without touching real infrastructure. apply executes those changes and updates the state file.

### Q2. How do you bring an existing, manually-created AWS resource under Terraform management?

Write a resource block matching its expected configuration, then run terraform import to link it into the state file, then run plan to confirm no diff.

### Q3. What replaced terraform taint in modern Terraform?

The -replace flag on terraform plan/apply, e.g. terraform apply -replace=aws\_instance.web, which is safer because it shows the replacement in the plan before executing.

### Q4. How do you detect infrastructure drift?

terraform plan itself shows drift as part of its diff. For a read-only check, terraform apply -refresh-only reconciles state with reality without changing resources.

---

# Chapter 5: Terraform Language Basics (HCL)

HCL (HashiCorp Configuration Language) is designed to be both human-readable and machine-parseable. Every .tf file is built from blocks, arguments, and expressions.

## Basic Block Anatomy

`anatomy.tf`

```
resource "aws_instance" "web" { # <block type> "<label>" "<label>"
  ami = "ami-0c94855ba95c71c99" # argument = value
  instance_type = "t3.micro"

  tags = { # nested block (map)
    Name = "web-server"
  }
}
```

## Comments

`comments.tf`

```
# Single line comment (preferred style)
// Also a single line comment
/*
Multi-line comment block
*/
```

## Variables, Outputs, and Locals (preview)

These three get full chapters of their own (6, 7, 8), but at a glance: variables are inputs, outputs are the values you expose after apply, and locals are named expressions you reuse internally.

`preview.tf`

```
variable "environment" {
  type = string
  default = "dev"
}

locals {
  name_prefix = "eFusion-${var.environment}"
}

output "prefix_used" {
  value = local.name_prefix
}
```

---

## Data Types

| Type          | Example                     | Notes                               |
|---------------|-----------------------------|-------------------------------------|
| string        | "hello"                     | Text values, supports interpolation |
| number        | 42, 3.14                    | Integers and floats                 |
| bool          | true / false                | Used in conditionals and count      |
| list(type)    | ["a","b","c"]               | Ordered collection, index from 0    |
| set(type)     | toiset(["a","b"])           | Unordered, unique values            |
| map(type)     | { key = "value" }           | Key-value pairs                     |
| object({...}) | { name=string, age=number } | Structured, typed attributes        |
| tuple(...)    | [string, number, bool]      | Fixed-length, mixed-type sequence   |

## Expressions and Operators

`expressions.tf`

```
# String interpolation
name = "server-${var.environment}"

# Arithmetic operators
disk_size = var.base_size * 2

# Comparison / logical operators
is_prod = var.environment == "production"
allow = var.enabled && !var.maintenance_mode

# Conditional (ternary) expression
instance_type = var.environment == "production" ? "m5.large" : "t3.micro"

# For expression (transforms a collection)
upper_names = [for n in var.names : upper(n)]
```

---

## Chapter Summary

- HCL files are built from blocks (type, labels, arguments) and expressions.
- Terraform supports primitive types (string, number, bool) and collection types (list, set, map, object).
- Interpolation, conditionals, and for-expressions let you write dynamic, DRY configuration.

## Best Practices

- 
- Run terraform fmt before every commit so blocks stay canonically indented.
  - Prefer explicit typed variables over untyped ones to catch mistakes at plan time.
  - Use for-expressions instead of copy-pasting near-identical resource blocks.

## Common Mistakes

- Mixing tabs and spaces (terraform fmt fixes this automatically - just run it).
- Using a list where a set was intended, causing unnecessary reordering diffs.
- Forgetting that HCL string interpolation requires `${...}` only inside strings, not at top level.

## Interview Questions

### Q1. What are the building blocks of an HCL file?

Blocks (with a type and optional labels), arguments (key = value pairs), and expressions (references, operators, functions) that compute those values.

### Q2. What is the difference between a list, a set, and a map in Terraform?

A list is an ordered, possibly-duplicated collection indexed by number. A set is unordered with unique values. A map is a set of key-value pairs accessed by key.

### Q3. How do you write a conditional value in Terraform?

Using the ternary operator: `condition ? true_val : false_val`, e.g. `var.env == "prod" ? "m5.large" : "t3.micro"`.

---

## PART II - CORE LANGUAGE

# Chapter 6: Variables

Variables are how you parameterize Terraform code so the same module can build a dev environment today and a production environment tomorrow, just by changing input values.

## Input Variable Declaration

`variables.tf`

```
variable "instance_type" {
  type = string
  description = "EC2 instance type for the web tier"
  default = "t3.micro"
}

variable "environment" {
  type = string
  validation {
    condition = contains(["dev", "qa", "uat", "production"], var.environment)
    error_message = "environment must be one of dev, qa, uat, production."
  }
}
```

## Variable Types

`types.tf`

```
variable "subnet_ids" { type = list(string) }
variable "tags" { type = map(string) }
variable "enable_https" { type = bool }
variable "az_count" { type = number }

variable "vpc_config" {
  type = object({
    cidr_block = string
    az_count = number
  })
}
```

## Sensitive Variables

`sensitive.tf`

```
variable "db_password" {
  type = string
  sensitive = true # hides value from plan/apply CLI output and logs
}
```

**WARNING:** sensitive = true hides values from CLI output, but the raw value is still stored in the state file. Always encrypt your state backend (S3 + KMS) and never commit secrets directly in .tfvars files to Git.

## Passing Values: CLI, Environment Variables, tfvars

```
# 1. Command line flag
terraform apply -var="instance_type=t3.large"

# 2. Environment variable (prefix TF_VAR_)
export TF_VAR_instance_type=t3.large

# 3. A .tfvars file
terraform apply -var-file="prod.tfvars"

# 4. Auto-loaded tfvars (no flag needed)
# terraform.tfvars - always auto-loaded
# *.auto.tfvars - always auto-loaded, any matching filename
```

### prod.tfvars

```
# prod.tfvars
environment = "production"
instance_type = "m5.large"
tags = {
  Team = "eFusion-Platform"
}
```

## Chapter Summary

- Variables parameterize configuration for reuse across environments.
- Terraform supports typed primitives, collections, and structured objects for variables.
- Values can be supplied via CLI flags, environment variables, or .tfvars files, in a defined precedence order.

## Best Practices

- Always add type and description to every variable for self-documenting code.
- Use validation blocks to fail fast on bad input instead of a partial, broken apply.
- Keep one .tfvars file per environment (dev.tfvars, prod.tfvars) instead of branching code.

---

## Common Mistakes

- Committing a prod.tfvars file with real secrets to a public Git repository.
- Forgetting that CLI -var flags override tfvars files, which override defaults - causing confusion.
- Marking a variable sensitive and assuming that also encrypts it inside the state file (it does not).

## Interview Questions

### Q1. What is the variable precedence order in Terraform?

From lowest to highest priority: defaults in variable blocks, environment variables (TF\_VAR\_\*), terraform.tfvars, \*.auto.tfvars (alphabetically), then -var and -var-file flags on the command line, with the last -var on the command line winning.

### Q2. How do you mark a variable as sensitive and what does that actually protect?

Set sensitive = true in the variable block. It suppresses the value from plan/apply console output and error messages, but the value is still stored in plaintext inside the state file, so state encryption is still required.

### Q3. How would you validate that an input variable only accepts specific values?

Using a validation block with a condition expression (e.g., contains(..., var.x)) and a custom error\_message.

---

# Chapter 7: Outputs

Outputs expose values from your configuration after apply - IDs, IPs, ARNs - so other modules, pipelines, or engineers can consume them without digging through the state file.

`outputs.tf`

```
output "vpc_id" {
  description = "ID of the created VPC"
  value = aws_vpc.main.id
}

output "db_password" {
  value = aws_db_instance.main.password
  sensitive = true
}

output "alb_dns_name" {
  value = aws_lb.app.dns_name
}
```

## Consuming Outputs

```
# CLI
terraform output
terraform output -raw vpc_id
terraform output -json > outputs.json

# From another module (root module referencing a child module)
module "network" {
  source = "../modules/network"
}
resource "aws_instance" "web" {
  subnet_id = module.network.public_subnet_id
}
```

**TIP:** Outputs are also how CI/CD pipelines pass data between stages - e.g. a 'network' pipeline stage outputs `vpc_id` and `subnet_ids`, and a downstream 'compute' stage's `terraform_remote_state` data source reads them.

---

## Chapter Summary

- Outputs expose selected resource attributes after apply for humans, pipelines, or other modules.
- Outputs can be marked sensitive to hide them from default CLI output.
- Child module outputs are the primary way root configurations consume module results.

---

## Best Practices

- Always add a description to outputs meant for cross-team or cross-module consumption.
- Output only what's needed - avoid dumping entire resource objects.
- Use terraform output -json in automation scripts instead of parsing human-readable text.

## Common Mistakes

- Forgetting to mark password/secret outputs as sensitive.
- Referencing module outputs before the module block name matches the actual module label.

## Interview Questions

### Q1. What is the purpose of an output block?

To expose specific values (like a resource ID or IP) from a Terraform configuration for use by operators, other modules, or automation pipelines.

### Q2. How do you consume the output of a child module in the root module?

Reference it as `module...`, e.g. `module.network.vpc_id`.

---

# Chapter 8: Locals

Locals are named, computed values, private to the module. They exist purely to keep your code DRY and readable - they are not inputs (like variables) and not exported (like outputs).

## Why Locals

- Avoid repeating the same expression (a naming convention, a tag map) across dozens of resources.
- Give a complex expression a meaningful name instead of inlining it everywhere.
- Centralize a computed value so changing it in one place updates every usage.

`locals.tf`

```
locals {
  name_prefix = "efusion-${var.environment}"

  common_tags = {
    Project = "eFusion"
    Environment = var.environment
    ManagedBy = "Terraform"
  }

  # Computed based on environment
  instance_count = var.environment == "production" ? 6 : 2
}

resource "aws_instance" "app" {
  count = local.instance_count
  ami = var.ami_id
  instance_type = var.instance_type
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-${count.index}" })
}
```

**TIP:** On the eFusion-style platform work, a single locals block defining `common_tags` and `name_prefix`, reused across every module, is the single highest-leverage pattern for consistent tagging and cost allocation across dozens of AWS accounts.

---

## Chapter Summary

- Locals are private, computed, reusable values scoped to a single module.
- They reduce duplication and centralize naming/tagging conventions.
- Unlike variables, locals cannot be set from outside the module.

## Best Practices

- Define a `common_tags` local in every module and `merge()` it into every resource's tags.

- 
- Keep locals simple and readable - if a local needs a comment to explain it, consider a function/module instead.

## Common Mistakes

- Confusing locals with variables and trying to pass a value into a local from the CLI (not possible).
- Creating circular references between locals (Terraform will error at plan time).

## Interview Questions

### Q1. What is the difference between a variable and a local value?

A variable is an external input that can be set by the caller (CLI, tfvars, module block). A local is an internal, computed value, private to the module, used purely to avoid repeating expressions.

### Q2. Give a real-world use case for locals.

Defining a `common_tags` map once and merging it into every resource's `tags` argument, ensuring consistent tagging for cost allocation across an entire AWS account.

---

# Chapter 9: Providers

A provider block configures how Terraform authenticates and communicates with a specific API. You can configure multiple providers, and even multiple instances of the same provider (for multi-region or multi-account setups).

## AWS Provider

`providers.tf`

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
  profile = "default" # or use env vars / IAM role
}

# A second, aliased provider for a different region
provider "aws" {
  alias = "us_east"
  region = "us-east-1"
}

resource "aws_instance" "primary" {
  provider = aws # default provider
  # ...
}

resource "aws_acm_certificate" "cf_cert" {
  provider = aws.us_east # CloudFront certs must be in us-east-1
  # ...
}
```

## Azure and Google Providers (for comparison)

```
# Azure
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}

# Google Cloud
provider "google" {
  project = "my-gcp-project"
  region = "asia-south1"
}
```

## Version Constraints

| Constraint | Meaning                                               |
|------------|-------------------------------------------------------|
| = 5.31.0   | Exact version only                                    |
| != 5.20.0  | Any version except this one                           |
| > 5.0.0    | Greater than                                          |
| ~> 5.0     | Any 5.x version (pessimistic constraint, most common) |
| ~> 5.31.0  | Any 5.31.x patch version only                         |

## Chapter Summary

- Providers are the bridge between HCL and a real API - AWS, Azure, GCP, and thousands more.
- Provider aliasing supports multi-region and multi-account architectures in one configuration.
- Version constraints in `required_providers` prevent unexpected breaking upgrades.

## Best Practices

- Always pin providers with `~>` to get patch fixes without unexpected major-version breakage.
- Use provider aliases for cross-region resources like CloudFront ACM certs (which must be `us-east-1`).
- Prefer IAM roles / OIDC federation over static access keys for provider authentication in CI/CD.

## Common Mistakes

- Hardcoding AWS access keys directly in provider blocks (a major security anti-pattern).
- Forgetting the `provider = aws.alias` argument on a resource that needs a non-default provider.
- Not pinning provider versions, leading to 'works on my machine' drift between engineers.

---

## Interview Questions

### Q1. How do you manage resources across two AWS regions in a single Terraform configuration?

Declare a second aws provider block with a unique alias and matching region, then set provider = aws.alias\_name on any resource that needs to be created in that region.

### Q2. Why should you avoid hardcoding AWS credentials in a provider block?

It's a severe security risk if committed to Git, and it doesn't support credential rotation. Best practice is to rely on the AWS credential chain (IAM roles, environment variables, or an AWS SSO/OIDC federated identity) instead.

### Q3. What does the ~> version constraint mean?

The pessimistic constraint operator: ~> 5.0 allows any 5.x version, while ~> 5.31.0 locks to the 5.31.x patch series only, allowing bug-fix upgrades but not new minor/major versions.

---

# Chapter 10: Resources

Resources are the most important block type in Terraform - each one represents a real infrastructure object: an EC2 instance, an S3 bucket, an IAM role.

## Resource Block Anatomy

`resource_anatomy.tf`

```
resource "aws_instance" "web" {
  ami = "ami-0c94855ba95c71c99"
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true
    prevent_destroy = false
    ignore_changes = [tags["LastDeployedBy"]]
  }
}
```

## Implicit and Explicit Dependencies

```
# Implicit dependency: Terraform infers order from the reference
resource "aws_subnet" "public" {
  vpc_id = aws_vpc.main.id # depends on aws_vpc.main automatically
}

# Explicit dependency: when there's no direct attribute reference
resource "aws_instance" "app" {
  # ...
  depends_on = [aws_iam_role_policy_attachment.app_policy]
}
```

## count vs for\_each

```
# count - simple numeric repetition
resource "aws_instance" "web" {
  count = 3
  ami = var.ami_id
  instance_type = "t3.micro"
  tags = { Name = "web-${count.index}" }
}

# for_each - keyed repetition (preferred for named, distinct resources)
resource "aws_instance" "app" {
  for_each = toset(["api", "worker", "scheduler"])
  ami = var.ami_id
  instance_type = "t3.micro"
  tags = { Name = "app-${each.key}" }
}
```

**TIP:** Prefer `for_each` over `count` when resources are conceptually distinct (like named services). With `count`, removing an item from the middle of a list shifts every subsequent index, forcing Terraform to destroy and recreate resources unnecessarily. `for_each` keys by name, so removing one item only affects that item.

## Real-World AWS Example: Web Server with Security Group

`web_server.tf`

```
resource "aws_security_group" "web_sg" {
  name = "${local.name_prefix}-web-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 443
    to_port = 443
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = local.common_tags
}

resource "aws_instance" "web" {
  ami = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
  subnet_id = aws_subnet.public[0].id
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  tags = merge(local.common_tags, { Name = "${local.name_prefix}-web" })
}
```

---

## Chapter Summary

- Resources represent real infrastructure objects and are the core unit of Terraform configuration.
- Terraform infers dependency order automatically from attribute references (implicit), or via `depends_on` (explicit).
- `count` is for simple numeric repetition; `for_each` is preferred for named, distinct resource sets.

## Best Practices

- Use `for_each` with `toset()/tomap()` for anything that has a natural, stable key.
- Use `lifecycle.prevent_destroy = true` on critical resources like production databases.
- Use `create_before_destroy` for zero-downtime replacement of resources like launch templates.

## Common Mistakes

- Using `count` for named services, causing full recreation cascades when the list order changes.
- Forgetting `depends_on` when a dependency exists only through IAM policy timing, not a direct attribute reference.

- 
- Setting `prevent_destroy = true` and then being unable to terraform destroy without first removing that lifecycle block.

## Interview Questions

### Q1. When would you choose `for_each` over `count`?

When resources represent distinct, named entities (like different microservices or named subnets). `for_each` keys resources by a stable identifier, so adding/removing one entry doesn't force recreation of unrelated resources, unlike `count` which reindexes by position.

### Q2. What is the difference between implicit and explicit dependencies?

Implicit dependencies are inferred automatically when one resource's argument references another resource's attribute. Explicit dependencies are declared with `depends_on` when no direct attribute reference exists but an ordering requirement still does (e.g., IAM eventual consistency).

### Q3. What does the lifecycle block let you control?

`create_before_destroy` (provision the replacement before destroying the original), `prevent_destroy` (block destroy operations), and `ignore_changes` (tell Terraform to ignore drift on specific attributes).

---

# Chapter 11: Data Sources

A data source lets Terraform read information about existing infrastructure that it does not manage - an AMI ID, an existing VPC, a Route53 zone, a secret from AWS Secrets Manager - and use it in your configuration.

`data_sources.tf`

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name = "name"
    values = ["al2023-ami-*-*x86_64"]
  }
}

data "aws_vpc" "existing" {
  filter {
    name = "tag:Name"
    values = ["shared-network-vpc"]
  }
}

resource "aws_instance" "web" {
  ami = data.aws_ami.amazon_linux.id
  subnet_id = data.aws_vpc.existing.id
}
```

## Production Use Case: Reading a Shared Secret

`secrets.tf`

```
data "aws_secretsmanager_secret_version" "db_creds" {
  secret_id = "efusion/prod/db-credentials"
}

locals {
  db_creds = jsondecode(data.aws_secretsmanager_secret_version.db_creds.secret_string)
}

resource "aws_db_instance" "main" {
  username = local.db_creds.username
  password = local.db_creds.password
}
```

**NOTE:** Data sources are read-only and evaluated during plan, so they never appear as 'create/change/destroy' - but they DO cause an API call every plan, which matters for rate-limited APIs at scale.

---

## Chapter Summary

- Data sources let Terraform read existing infrastructure it doesn't own or manage.
- Common production uses: dynamic AMI lookups, reading shared VPCs, and pulling secrets at plan time.
- Data sources are read-only - they never create, modify, or destroy anything.

## Best Practices

- Use data sources for AMI lookups instead of hardcoding AMI IDs that go stale.
- Combine data sources with `jsondecode()` to consume structured secrets cleanly.

## Common Mistakes

- Confusing a data source with a resource of the same name (data "aws\_vpc" vs resource "aws\_vpc").
- Not filtering data source queries tightly enough, causing 'multiple results found' errors.

## Interview Questions

### Q1. What is the difference between a resource and a data source?

A resource is something Terraform creates, updates, and destroys. A data source is read-only - it queries information about infrastructure that already exists, whether or not Terraform manages it.

### Q2. Give a real production use case for a data source.

Dynamically looking up the latest Amazon Linux AMI ID at plan time instead of hardcoding an AMI ID that becomes outdated and eventually gets deprecated by AWS.

---

# Chapter 12: Expressions

Expressions are how Terraform computes values - from simple string interpolation to complex conditional logic and iteration.

## Interpolation

```
name = "${var.project}-${var.environment}-web"
# Modern Terraform also allows direct references without ${} when the whole value is
the reference:
count = var.instance_count
```

## Conditional Expressions

```
instance_type = var.environment == "production" ? "m5.large" : "t3.micro"
subnet_count = length(var.azs) > 2 ? 3 : length(var.azs)
```

## For Expressions

```
# Transform a list
upper_azs = [for az in var.azs : upper(az)]

# Transform into a map
ami_by_region = { for r in var.regions : r => data.aws_ami.by_region[r].id }

# Filter while transforming
prod_only = [for e in var.environments : e if e == "production"]
```

## Splat Expressions

```
# Get all instance IDs from a list of resources created with count
instance_ids = aws_instance.web[*].id

# Equivalent to:
instance_ids = [for i in aws_instance.web : i.id]
```

---

## Chapter Summary

- Interpolation embeds expressions inside strings using `\${...}`.
- Conditional (ternary) expressions choose between two values based on a boolean condition.

- 
- For-expressions transform and filter lists/maps in a single line, replacing manual loops.

## Best Practices

- Prefer for-expressions over duplicating near-identical resource blocks.
- Keep conditional expressions readable - extract complex conditions into a named local first.

## Common Mistakes

- Overusing deeply nested ternary expressions that become unreadable.
- Forgetting splat expressions (`[*]`) return a list even when count = 1.

## Interview Questions

### Q1. What does a for expression do in Terraform?

It iterates over a list, set, or map, transforming (and optionally filtering) values into a new list or map, similar to a list comprehension in Python.

### Q2. What is a splat expression?

Syntax like `aws_instance.web[*].id` that extracts a given attribute from every instance of a resource created with `count` or `for_each`, returning it as a list.

# Chapter 13: Built-in Functions

Terraform has no user-defined functions, but ships over 100 built-in functions across strings, numbers, collections, encoding, and the filesystem. These are the ones you'll actually use in production.

| Category   | Function Call                                   | Result                   |
|------------|-------------------------------------------------|--------------------------|
| String     | <code>upper("abc")</code>                       | "ABC"                    |
| String     | <code>lower("ABC")</code>                       | "abc"                    |
| String     | <code>trim(" hi ", " ")</code>                  | "hi"                     |
| String     | <code>join("-", ["a","b"])</code>               | "a-b"                    |
| String     | <code>split(", ", "a,b,c")</code>               | ["a","b","c"]            |
| String     | <code>format("%s-%d", "web", 3)</code>          | "web-3"                  |
| String     | <code>replace("foo-bar", "-", "_")</code>       | "foo_bar"                |
| Numeric    | <code>max(3, 7, 2)</code>                       | 7                        |
| Numeric    | <code>min(3, 7, 2)</code>                       | 2                        |
| Numeric    | <code>ceil(4.1)</code>                          | 5                        |
| Numeric    | <code>floor(4.9)</code>                         | 4                        |
| Collection | <code>length([1,2,3])</code>                    | 3                        |
| Collection | <code>merge({a=1},{b=2})</code>                 | {a=1, b=2}               |
| Collection | <code>concat([1,2],[3,4])</code>                | [1,2,3,4]                |
| Collection | <code>distinct([1,1,2,3])</code>                | [1,2,3]                  |
| Collection | <code>lookup({a=1}, "a", 0)</code>              | 1                        |
| Collection | <code>contains(["a","b"], "a")</code>           | true                     |
| Encoding   | <code>jsonencode({a=1})</code>                  | '{"a":1}'                |
| Encoding   | <code>jsondecode('{"a":1}')</code>              | {a=1}                    |
| Encoding   | <code>base64encode("hi")</code>                 | "aGk="                   |
| Filesystem | <code>file("./id_rsa.pub")</code>               | contents of the file     |
| Filesystem | <code>templatefile("tpl.tftpl", {n="x"})</code> | rendered template string |

## Real Example: `templatefile()` for a User Data Script

---

### templatefile-example

```
# user_data.tftpl
#!/bin/bash
echo "Environment: ${environment}" >> /var/log/bootstrap.log
systemctl start ${service_name}

# main.tf
resource "aws_instance" "app" {
  user_data = templatefile("${path.module}/user_data.tftpl", {
    environment = var.environment
    service_name = "eFusion-agent"
  })
}
```

---

## Chapter Summary

- Terraform ships 100+ built-in functions; Terraform does not support user-defined functions.
- String, numeric, collection, encoding, and filesystem functions cover almost every real need.
- `templatefile()` is the standard way to render dynamic scripts (user\_data, configs) from a template file.

## Best Practices

- Use `merge()` to combine common\_tags with resource-specific tags, rather than duplicating maps.
- Use `jsonencode()/jsondecode()` instead of manually building JSON strings.
- Keep user\_data scripts in separate .tftpl files rendered via `templatefile()`, not inline heredocs.

## Common Mistakes

- Trying to write a custom function (not supported - use a module or a for-expression instead).
- Forgetting `templatefile()` requires all template variables to be explicitly passed in the second argument.

## Interview Questions

### Q1. Can you write custom functions in Terraform?

No. Terraform only supports its built-in function library. Custom reusable logic is achieved through modules, for-expressions, and locals instead.

### Q2. What does the `merge()` function do and when would you use it?

It combines two or more maps into one, with later maps overriding earlier keys on conflict. Commonly used to combine a shared common\_tags map with resource-specific tags.

### Q3. What is `templatefile()` used for?

Rendering an external template file (like a user\_data bootstrap script) with variable substitution, keeping large blocks of shell/config script out of the .tf file itself.

---

# Chapter 14: Meta-Arguments

Meta-arguments are special arguments that work on every resource type, controlling behavior rather than the resource's own configuration: `depends_on`, `count`, `for_each`, `provider`, and `lifecycle`.

`meta_arguments.tf`

```
resource "aws_instance" "app" {
  count = 2
  ami = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  provider = aws.us_east
  depends_on = [aws_iam_role_policy_attachment.app]

  lifecycle {
    create_before_destroy = true
    prevent_destroy = false
    ignore_changes = [tags["LastModifiedBy"], ami]
  }
}
```

## Deep Dive: Lifecycle Options

| Option                             | Effect                                                                                                                                      |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>create_before_destroy</code> | Provisions the replacement resource before destroying the old one - critical for zero-downtime replacements like launch templates and ASGs. |
| <code>prevent_destroy</code>       | Terraform errors out on any plan/apply that would destroy this resource. Common on production RDS and S3 buckets.                           |
| <code>ignore_changes</code>        | Tells Terraform to ignore drift on specific attributes (e.g., tags set by an external autoscaler).                                          |

**WARNING:** `prevent_destroy` is enforced at plan time and blocks terraform destroy entirely for that resource. To genuinely decommission a protected resource, you must first remove the lifecycle block, then destroy.

---

## Chapter Summary

- Meta-arguments (`depends_on`, `count`, `for_each`, `provider`, `lifecycle`) apply to any resource type.
- `lifecycle` controls replacement order, destroy protection, and drift suppression.
- `create_before_destroy` is essential for zero-downtime infrastructure replacement patterns.

## Best Practices

- 
- Set `prevent_destroy = true` on production databases and state-holding S3 buckets.
  - Use `ignore_changes` selectively - ignoring too much hides real drift you should know about.

## Common Mistakes

- Setting `create_before_destroy` on a resource with a globally unique name, causing a naming conflict during creation.
- Using `ignore_changes = all` as a blanket fix, which silently hides all future configuration drift.

## Interview Questions

### Q1. What does `create_before_destroy` actually solve?

It avoids downtime and 'resource already exists' failures when replacing something like a launch template or ASG by ensuring the new resource is created and validated before the old one is torn down.

### Q2. Why would you use `ignore_changes` on tags?

Because an external system (like an autoscaling process or a security scanner) might modify tags outside Terraform. `ignore_changes` prevents Terraform from fighting that external process every apply.

---

# Chapter 15: Dynamic Blocks

Some resources require repeated nested blocks (like ingress rules in a security group). Instead of copy-pasting a block for every rule, dynamic blocks generate them from a list or map at plan time.

`dynamic_blocks.tf`

```
variable "ingress_rules" {
  type = list(object({
    port = number
    protocol = string
    cidr_blocks = list(string)
  }))
  default = [
    { port = 443, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] },
    { port = 22, protocol = "tcp", cidr_blocks = ["10.0.0.0/8"] },
  ]
}

resource "aws_security_group" "web" {
  name = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port = ingress.value.port
      to_port = ingress.value.port
      protocol = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}
```

**NOTE:** Inside a dynamic block, the iterator variable (named after the block type by default, 'ingress' here) exposes `.key` and `.value`, exactly like a `for_each` on a resource.

---

## Chapter Summary

- Dynamic blocks generate repeated nested configuration blocks from a list or map input.
- They eliminate copy-pasted nested blocks like security group rules or IAM policy statements.
- The iterator inside a dynamic block behaves like `for_each`: it exposes `.key` and `.value`.

## Best Practices

- Use dynamic blocks for variable-length nested configuration (security group rules, EBS block devices).
- Keep the input variable's object type well-defined so consumers get validation for free.

---

## Common Mistakes

- Using dynamic blocks for a fixed, small number of nested blocks where a plain static block reads clearer.
- Forgetting the content block must match the nested block's exact schema (e.g., all required ingress fields).

## Interview Questions

### **Q1. When would you use a dynamic block instead of a static nested block?**

When the number of nested blocks (like ingress rules in a security group) is variable and driven by input data, rather than fixed and known at code-writing time.

### **Q2. What does the iterator expose inside a dynamic block's content block?**

It exposes `.key` and `.value` for the current item being iterated, just like a `for_each` expression on a resource.

---

## PART III - SCALING TERRAFORM

# Chapter 16: Modules

A module is simply a directory of .tf files. Every configuration has at least one module - the root module. Real production Terraform is organized into small, reusable, versioned child modules.

## Why Modules

- **Reusability:** Write a VPC module once, use it for dev, QA, UAT, and production.
- **Encapsulation:** Hide implementation complexity behind a clean variable/output interface.
- **Testability:** Small modules are easier to review, test, and version independently.
- **Team ownership:** Platform teams publish modules; application teams consume them.

## Standard Module Structure

Fig 16.1 - Standard Module Layout

```
modules/vpc/  
|-- main.tf # resources  
|-- variables.tf # inputs  
|-- outputs.tf # exports  
|-- versions.tf # provider requirements  
\-- README.md # usage docs
```

### modules/vpc

```
# modules/vpc/variables.tf  
variable "cidr_block" { type = string }  
variable "azs" { type = list(string) }  
variable "name_prefix" { type = string }  
  
# modules/vpc/main.tf  
resource "aws_vpc" "this" {  
  cidr_block = var.cidr_block  
  tags = { Name = "${var.name_prefix}-vpc" }  
}  
  
resource "aws_subnet" "public" {  
  for_each = toset(var.azs)  
  vpc_id = aws_vpc.this.id  
  cidr_block = cidrsubnet(var.cidr_block, 8, index(var.azs, each.key))  
  availability_zone = each.key  
}  
  
# modules/vpc/outputs.tf  
output "vpc_id" { value = aws_vpc.this.id }  
output "public_subnet_ids" { value = [for s in aws_subnet.public : s.id] }
```

---

## Calling a Module

calling\_modules.tf

```
module "vpc" {
  source = "../modules/vpc"
  cidr_block = "10.0.0.0/16"
  azs = ["ap-south-1a", "ap-south-1b"]
  name_prefix = "efusion-prod"
}

resource "aws_instance" "web" {
  subnet_id = module.vpc.public_subnet_ids[0]
}

# Calling a module from a Git tag (versioned, production standard)
module "vpc_versioned" {
  source = "git::https://github.com/org/tf-modules.git//vpc?ref=v1.4.0"
}

# Calling a module from the public Terraform Registry
module "vpc_registry" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.8.1"
}
```

**WARNING:** Production standard: never reference a module by a mutable branch (?ref=main). Always pin to a Git tag or a released version number, so a change in the module repo can never silently break every environment that consumes it.

---

## Chapter Summary

- Every Terraform configuration is a module; the root module can call reusable child modules.
- Modules encapsulate resources behind a clean variables-in, outputs-out interface.
- Production modules should be sourced from versioned Git tags or the Terraform Registry, never a floating branch.

## Best Practices

- Keep modules small and single-purpose (vpc, eks-cluster, rds, alb) rather than one giant module.
- Document every module's inputs/outputs in a README.md.
- Pin module sources to an exact version/tag for reproducible builds.

## Common Mistakes

- Building one monolithic module that tries to create an entire environment (hard to reuse or test).

- 
- Referencing a module by a mutable branch name, causing unexpected changes across environments.
  - Not exposing enough outputs from a module, forcing consumers to duplicate resource lookups.

## Interview Questions

### Q1. What is a Terraform module?

A self-contained directory of .tf files with its own variables and outputs, callable from a root or other module, used to encapsulate and reuse infrastructure patterns.

### Q2. How do you version-pin a module sourced from Git?

By appending ?ref= to the Git source URL, e.g. `git::https://github.com/org/modules.git/vpc?ref=v1.4.0`, pointing to an immutable tag rather than a branch.

### Q3. What's the risk of referencing a module by a branch instead of a tag?

Any future commit to that branch changes the module's behavior for every consumer on their next terraform init/apply, without an explicit, reviewed version bump - a serious reproducibility risk.

---

# Chapter 17: Remote State

By default, Terraform stores state locally in `terraform.tfstate`. That's fine for a solo experiment - it's unacceptable for a team. Remote state stores the state file in a shared, durable, lockable backend.

## S3 Backend with DynamoDB Locking (the AWS standard)

`backend.tf`

```
terraform {
  backend "s3" {
    bucket = "efusion-terraform-state-prod"
    key    = "network/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt = true
    kms_key_id = "arn:aws:kms:ap-south-1:111122223333:key/abcd-1234"
  }
}
```

`bootstrap_backend.tf`

```
# Provisioning the backend itself (chicken-and-egg: usually done once, manually or in a
bootstrap stack)
resource "aws_s3_bucket" "tf_state" {
  bucket = "efusion-terraform-state-prod"
  lifecycle { prevent_destroy = true }
}

resource "aws_s3_bucket_versioning" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_dynamodb_table" "tf_locks" {
  name = "terraform-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

## Reading Another Stack's State (`terraform_remote_state`)

`remote_state_data.tf`

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "efusion-terraform-state-prod"
    key    = "network/terraform.tfstate"
    region = "ap-south-1"
  }
}

resource "aws_instance" "app" {
  subnet_id = data.terraform_remote_state.network.outputs.public_subnet_id
}
```

**NOTE:** With Terraform 1.6+, HCP Terraform / Terraform Cloud offer a managed state backend with built-in locking, encryption, and RBAC - removing the need to self-host the S3+DynamoDB pattern, though the S3 pattern remains extremely common in bank/enterprise environments with strict data-residency requirements.

## Chapter Summary

- Remote state stores terraform.tfstate in a shared, durable, team-accessible backend.
- S3 + DynamoDB is the classic AWS-native backend: S3 for storage, DynamoDB for state locking.
- terraform\_remote\_state lets one stack safely read another stack's outputs.

## Best Practices

- Always enable S3 bucket versioning on the state bucket - it's your rollback safety net.
- Enable encryption (KMS) on the state bucket; state often contains secrets in plaintext.
- Use a separate state key per stack/environment (network/, compute/, database/) rather than one giant state file.

## Common Mistakes

- Committing terraform.tfstate to Git (it often contains secrets and is a merge-conflict nightmare).
- Sharing one state file across every environment, so a mistake in dev can affect production's plan.
- Forgetting DynamoDB locking, leading to two engineers running apply simultaneously and corrupting state.

## Interview Questions

### Q1. Why is local state unsuitable for team environments?

It isn't shared, isn't locked (two people can apply concurrently and corrupt it), and isn't durable or encrypted by default - remote state with locking solves all three problems.

### Q2. What role does the DynamoDB table play in the S3 backend?

---

It provides state locking: before any operation, Terraform writes a lock record to DynamoDB so no other terraform process can run a conflicting operation concurrently.

**Q3. How can one Terraform stack consume the outputs of another stack?**

Using the `terraform_remote_state` data source pointed at the other stack's backend configuration and state key, then referencing `data.terraform_remote_state.X.outputs.Y`.

---

# Chapter 18: State File Deep Dive

terraform.tfstate is a JSON document. Understanding its structure and the state subcommands is essential for real-world troubleshooting, refactors, and incident response.

terraform.tfstate (simplified)

```
{
  "version": 4,
  "terraform_version": "1.9.0",
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "instances": [
        { "attributes": { "id": "i-0abc123", "ami": "ami-0c94855ba95c71c99" } }
      ]
    }
  ]
}
```

## State Commands

```
terraform state list # list every resource address in state
terraform state show aws_instance.web # show full attributes of one resource
terraform state mv aws_instance.old aws_instance.new # rename without destroy/recreate
terraform state rm aws_instance.web # stop tracking (does NOT delete real resource)
terraform state pull > backup.tfstate # download raw state
terraform state push backup.tfstate # upload raw state (dangerous - use carefully)
```

## State Locking

When a backend supports locking (S3+DynamoDB, HCP Terraform), Terraform acquires a lock before any plan/apply/destroy and releases it after. If a process crashes mid-run, use terraform force-unlock to manually clear a stuck lock - but only after confirming no other process is actually running.

## Importing Existing Resources

```
# 1. Write a resource block matching the real resource
resource "aws_s3_bucket" "logs" {
  bucket = "efusion-prod-logs"
}

# 2. Import it into state
terraform import aws_s3_bucket.logs efusion-prod-logs

# 3. Run plan to confirm zero diff (adjust the resource block until it matches exactly)
terraform plan
```

## Moving and Removing State

```
# Renaming a resource without destroying it
terraform state mv aws_instance.web aws_instance.app_server

# Moving a resource into a module
terraform state mv aws_instance.web module.compute.aws_instance.web

# Removing a resource from state (e.g., before deleting it manually outside Terraform)
terraform state rm aws_s3_bucket.old_logs
```

**TIP:** terraform state rm does NOT delete the real cloud resource - it only makes Terraform forget about it. This is the correct tool when handing a resource off to be managed by a different team or tool.

## Handling Sensitive Data in State

- State can contain plaintext secrets (DB passwords, private keys) even for variables marked sensitive.
- Always enable encryption at rest on the state backend (S3 SSE-KMS).
- Restrict IAM/RBAC access to the state bucket/table to the smallest group of engineers possible.
- Consider a secrets manager (AWS Secrets Manager) and reference secrets via data sources instead of raw variables where possible.

## Chapter Summary

- The state file is JSON, tracks resource attributes, and is the ground truth Terraform diffs against.
- terraform state subcommands (list, show, mv, rm) let you inspect and refactor state without destroying resources.
- State locking prevents concurrent operations from corrupting the file; force-unlock clears stuck locks.
- State can contain plaintext secrets - encryption and access control on the backend are mandatory.

## Best Practices

- 
- Back up state (terraform state pull) before any risky manual state operation.
  - Use terraform state mv during refactors (e.g., moving a resource into a module) to avoid destroy/recreate.
  - Restrict state bucket access tightly - state readability is equivalent to secret readability.

## Common Mistakes

- Running terraform force-unlock without first confirming no other process is genuinely running.
- Manually editing the .tfstate JSON file by hand instead of using terraform state commands.
- Deleting a resource manually in the console and forgetting to terraform state rm it, causing the next apply to fail or attempt recreation.

## Interview Questions

### Q1. What happens if you manually delete a resource in the AWS Console that Terraform manages?

Terraform's state still references it. On the next plan/apply, Terraform detects it's gone and, depending on configuration, will either propose to recreate it or error - the state must be reconciled with terraform state rm or apply -refresh-only.

### Q2. How do you safely rename a resource in code without destroying and recreating it?

Use terraform state mv to update the state file's mapping to match the renamed resource block, then confirm terraform plan shows no changes.

### Q3. Why must a Terraform state backend be encrypted?

Because the state file can contain plaintext secrets (passwords, keys, tokens) even when the originating variable was marked sensitive - sensitive only hides console/log output, not state content.

---

# Chapter 19: Terraform Workspaces

Workspaces let a single Terraform configuration manage multiple, isolated state files - commonly used for dev/qa/uat/production, though many teams prefer separate directories/backends for stronger isolation.

`workspaces.tf`

```
terraform workspace list
terraform workspace new staging
terraform workspace select production
terraform workspace show

# Referencing the current workspace inside configuration
resource "aws_instance" "app" {
  instance_type = terraform.workspace == "production" ? "m5.large" : "t3.micro"
  tags = { Environment = terraform.workspace }
}
```

## Workspaces vs Separate Directories/Backends

| Approach                      | Pros                                                                            | Cons                                                                   |
|-------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Workspaces                    | Single codebase, quick to switch, less duplication                              | Shared backend config; easy to apply to the wrong workspace by mistake |
| Separate directories/backends | Strong isolation, separate state buckets/keys, separate IAM permissions per env | More files to maintain, some duplication                               |

**NOTE:** In BFSI/regulated environments, separate state backends per environment (with separate IAM roles) are usually preferred over workspaces, since production must be provably isolated at the credential level, not just the state-file level.

---

## Chapter Summary

- Workspaces allow one configuration to manage multiple isolated state files (dev, qa, uat, prod).
- `terraform.workspace` is available inside configuration to branch behavior per environment.
- Regulated environments often prefer fully separate backends/directories over workspaces for stronger isolation.

## Best Practices

- Always run `terraform workspace show` before apply, to confirm you're in the intended environment.

- 
- Use distinctly different variable files per workspace to avoid accidental cross-environment config bleed.

## Common Mistakes

- Forgetting which workspace is currently selected and applying dev-sized resources to production.
- Using workspaces as a substitute for proper IAM isolation between environments.

## Interview Questions

### Q1. What is a Terraform workspace?

A named, isolated state file within the same backend and configuration, letting you manage multiple environments (dev, qa, prod) from one codebase by switching the active workspace.

### Q2. Why might a bank or regulated enterprise avoid using workspaces for production isolation?

Because workspaces share the same backend and often the same IAM credentials; regulated environments typically require production to be isolated at the credential and account level, not just the state-file level, so separate backends/directories are safer.

---

# Chapter 20: Provisioners

Provisioners run scripts on a local machine or a remote resource as part of create/destroy. HashiCorp considers them a last resort - prefer AMI baking (Packer) or configuration management (Ansible) instead.

`provisioners.tf`

```
resource "aws_instance" "web" {
  ami = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  key_name = aws_key_pair.deployer.key_name

  # Runs a command on the machine executing Terraform
  provisioner "local-exec" {
    command = "echo ${self.public_ip} >> inventory.txt"
  }

  # Runs a command ON the remote instance over SSH
  provisioner "remote-exec" {
    inline = [
      "sudo yum install -y docker",
      "sudo systemctl start docker",
    ]
    connection {
      type = "ssh"
      user = "ec2-user"
      private_key = file("~/ssh/deployer.pem")
      host = self.public_ip
    }
  }

  # Copies a file to the remote instance
  provisioner "file" {
    source = "app-config.yml"
    destination = "/tmp/app-config.yml"
    connection {
      type = "ssh"
      user = "ec2-user"
      private_key = file("~/ssh/deployer.pem")
      host = self.public_ip
    }
  }
}
```

**WARNING:** Provisioners have no idempotency guarantee and cannot be tracked in a plan the way resource attributes can. Prefer baking configuration into an AMI with Packer, or using cloud-init/user\_data, or handing post-boot configuration to Ansible - reserve provisioners for narrow bootstrap use cases only.

---

## Chapter Summary

- local-exec runs a command on the machine running Terraform; remote-exec runs it on the target resource over SSH/WinRM.
- The file provisioner copies files to a remote resource before remote-exec runs.
- HashiCorp recommends provisioners as a last resort due to weak idempotency and plan visibility.

## Best Practices

- Prefer user\_data/cloud-init or a pre-baked AMI (Packer) over remote-exec provisioners.
- If you must use a provisioner, add on\_failure and timeout handling to avoid indefinitely hanging applies.

## Common Mistakes

- Relying on remote-exec for complex application deployment logic that really belongs in a CI/CD pipeline or Ansible.
- Not handling provisioner failures, causing 'tainted' resources that silently fail on the next apply.

## Interview Questions

### Q1. Why does HashiCorp recommend avoiding provisioners when possible?

Because they aren't idempotent, aren't visible in the plan diff the way resource attributes are, and can fail in ways Terraform can't cleanly retry - configuration management tools or pre-baked AMIs handle this more reliably.

### Q2. What's the difference between local-exec and remote-exec?

local-exec runs a command on the machine executing Terraform (your laptop or CI runner); remote-exec runs a command on the newly created remote resource itself, typically over SSH or WinRM.

---

## PART IV - AWS IN PRODUCTION

# Chapter 21: AWS Infrastructure Examples

This chapter is your reference library - complete, working Terraform code for every core AWS building block, the way you'd actually write it in production.

## 21.1 - VPC, Subnets, Internet Gateway, NAT Gateway, Route Tables

`vpc_network.tf`

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "${local.name_prefix}-vpc" }
}

resource "aws_subnet" "public" {
  for_each = { "ap-south-1a" = "10.0.1.0/24", "ap-south-1b" = "10.0.2.0/24" }
  vpc_id = aws_vpc.main.id
  cidr_block = each.value
  availability_zone = each.key
  map_public_ip_on_launch = true
  tags = { Name = "${local.name_prefix}-public-${each.key}" }
}

resource "aws_subnet" "private" {
  for_each = { "ap-south-1a" = "10.0.11.0/24", "ap-south-1b" = "10.0.12.0/24" }
  vpc_id = aws_vpc.main.id
  cidr_block = each.value
  availability_zone = each.key
  tags = { Name = "${local.name_prefix}-private-${each.key}" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = { Name = "${local.name_prefix}-igw" }
}

resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id = aws_subnet.public["ap-south-1a"].id
  tags = { Name = "${local.name_prefix}-nat" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
```

```
cidr_block = "0.0.0.0/0"
gateway_id = aws_internet_gateway.igw.id
}
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }
}

resource "aws_route_table_association" "public" {
  for_each = aws_subnet.public
  subnet_id = each.value.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  for_each = aws_subnet.private
  subnet_id = each.value.id
  route_table_id = aws_route_table.private.id
}
```

## 21.2 - Security Groups

`security_groups.tf`

```

resource "aws_security_group" "alb_sg" {
  name = "${local.name_prefix}-alb-sg"
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 443; to_port = 443; protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "app_sg" {
  name = "${local.name_prefix}-app-sg"
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 8080
    to_port = 8080
    protocol = "tcp"
    security_groups = [aws_security_group.alb_sg.id] # only ALB can reach app tier
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

## 21.3 - EC2 Instance

`ec2.tf`

```

resource "aws_instance" "app" {
  ami = data.aws_ami.al2023.id
  instance_type = "t3.medium"
  subnet_id = aws_subnet.private["ap-south-1a"].id
  vpc_security_group_ids = [aws_security_group.app_sg.id]
  iam_instance_profile = aws_iam_instance_profile.app.name

  root_block_device {
    volume_size = 30
    volume_type = "gp3"
    encrypted = true
  }

  tags = merge(local.common_tags, { Name = "${local.name_prefix}-app" })
}

```

## 21.4 - IAM Role and Policy

`iam.tf`

```
data "aws_iam_policy_document" "app_assume" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "app" {
  name = "${local.name_prefix}-app-role"
  assume_role_policy = data.aws_iam_policy_document.app_assume.json
}

resource "aws_iam_role_policy" "app_s3_read" {
  name = "s3-read-only"
  role = aws_iam_role.app.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["s3:GetObject", "s3:ListBucket"]
      Resource = [aws_s3_bucket.assets.arn, "${aws_s3_bucket.assets.arn}/*"]
    }]
  })
}

resource "aws_iam_instance_profile" "app" {
  name = "${local.name_prefix}-app-profile"
  role = aws_iam_role.app.name
}
```

## 21.5 - S3 Bucket

s3.tf

```

resource "aws_s3_bucket" "assets" {
  bucket = "${local.name_prefix}-assets"
}

resource "aws_s3_bucket_versioning" "assets" {
  bucket = aws_s3_bucket.assets.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "assets" {
  bucket = aws_s3_bucket.assets.id
  rule {
    apply_server_side_encryption_by_default { sse_algorithm = "aws:kms" }
  }
}

resource "aws_s3_bucket_public_access_block" "assets" {
  bucket = aws_s3_bucket.assets.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}

```

## 21.6 - RDS Database

`rds.tf`

```

resource "aws_db_subnet_group" "main" {
  name = "${local.name_prefix}-db-subnets"
  subnet_ids = [for s in aws_subnet.private : s.id]
}

resource "aws_db_instance" "main" {
  identifier = "${local.name_prefix}-db"
  engine = "postgres"
  engine_version = "16.3"
  instance_class = "db.t3.medium"
  allocated_storage = 100
  storage_encrypted = true
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db_sg.id]
  multi_az = true
  backup_retention_period = 7
  username = local.db_creds.username
  password = local.db_creds.password
  skip_final_snapshot = false
  final_snapshot_identifier = "${local.name_prefix}-db-final-snapshot"

  lifecycle { prevent_destroy = true }
}

```

---

## 21.7 - Application Load Balancer

alb.tf

```
resource "aws_lb" "app" {
  name = "${local.name_prefix}-alb"
  internal = false
  load_balancer_type = "application"
  subnets = [for s in aws_subnet.public : s.id]
  security_groups = [aws_security_group.alb_sg.id]
}

resource "aws_lb_target_group" "app" {
  name = "${local.name_prefix}-tg"
  port = 8080
  protocol = "HTTP"
  vpc_id = aws_vpc.main.id
  health_check {
    path = "/health"
    healthy_threshold = 2
    unhealthy_threshold = 3
    interval = 15
  }
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.app.arn
  port = 443
  protocol = "HTTPS"
  ssl_policy = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn = aws_acm_certificate.main.arn
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

## 21.8 - Auto Scaling Group

asg.tf

```
resource "aws_launch_template" "app" {
  name_prefix = "${local.name_prefix}-lt-"
  image_id = data.aws_ami.al2023.id
  instance_type = "t3.medium"
  iam_instance_profile { name = aws_iam_instance_profile.app.name }
  vpc_security_group_ids = [aws_security_group.app_sg.id]
  user_data = base64encode(templatefile("${path.module}/bootstrap.tftpl", { env =
  var.environment }))
  lifecycle { create_before_destroy = true }
}

resource "aws_autoscaling_group" "app" {
  desired_capacity = 2
  min_size = 2
  max_size = 6
  vpc_zone_identifier = [for s in aws_subnet.private : s.id]
  target_group_arns = [aws_lb_target_group.app.arn]

  launch_template {
    id = aws_launch_template.app.id
    version = "$Latest"
  }

  tag {
    key = "Name"
    value = "${local.name_prefix}-asg"
    propagate_at_launch = true
  }
}

resource "aws_autoscaling_policy" "scale_out" {
  name = "scale-out-on-cpu"
  autoscaling_group_name = aws_autoscaling_group.app.name
  policy_type = "TargetTrackingScaling"
  target_tracking_configuration {
    predefined_metric_specification { predefined_metric_type = "ASGAverageCPUUtilization" }
    target_value = 65.0
  }
}
```

## 21.9 - Lambda Function

`lambda.tf`

```

data "archive_file" "lambda_zip" {
  type = "zip"
  source_file = "${path.module}/src/handler.py"
  output_path = "${path.module}/build/handler.zip"
}

resource "aws_lambda_function" "processor" {
  function_name = "${local.name_prefix}-processor"
  filename = data.archive_file.lambda_zip.output_path
  source_code_hash = data.archive_file.lambda_zip.output_base64sha256
  handler = "handler.main"
  runtime = "python3.12"
  role = aws_iam_role.lambda_exec.arn
  timeout = 30
  memory_size = 256
  environment {
    variables = { ENVIRONMENT = var.environment }
  }
}

```

## 21.10 - CloudWatch Alarms and Log Groups

`cloudwatch.tf`

```

resource "aws_cloudwatch_log_group" "app" {
  name = "/efusion/${var.environment}/app"
  retention_in_days = 30
}

resource "aws_cloudwatch_metric_alarm" "high_cpu" {
  alarm_name = "${local.name_prefix}-high-cpu"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods = 2
  metric_name = "CPUUtilization"
  namespace = "AWS/EC2"
  period = 300
  statistic = "Average"
  threshold = 80
  alarm_actions = [aws_sns_topic.alerts.arn]
  dimensions = { AutoScalingGroupName = aws_autoscaling_group.app.name }
}

```

## 21.11 - ECR Repository

`ecr.tf`

```

resource "aws_ecr_repository" "app" {
  name = "${local.name_prefix}-app"
  image_tag_mutability = "IMMUTABLE"
  image_scanning_configuration { scan_on_push = true }
}

resource "aws_ecr_lifecycle_policy" "app" {
  repository = aws_ecr_repository.app.name
  policy = jsonencode({
    rules = [{
      rulePriority = 1
      description = "Expire untagged images after 14 days"
      selection = {
        tagStatus = "untagged"
        countType = "sinceImagePushed"
        countUnit = "days"
        countNumber = 14
      }
      action = { type = "expire" }
    }]
  })
}

```

## 21.12 - EKS Cluster

eks.tf

```

resource "aws_eks_cluster" "main" {
  name = "${local.name_prefix}-eks"
  role_arn = aws_iam_role.eks_cluster.arn
  version = "1.30"

  vpc_config {
    subnet_ids = concat([for s in aws_subnet.private : s.id], [for s in aws_subnet.public : s.id])
    endpoint_private_access = true
    endpoint_public_access = true
  }
}

resource "aws_eks_node_group" "main" {
  cluster_name = aws_eks_cluster.main.name
  node_group_name = "${local.name_prefix}-ng"
  node_role_arn = aws_iam_role.eks_nodes.arn
  subnet_ids = [for s in aws_subnet.private : s.id]
  instance_types = ["t3.large"]

  scaling_config {
    desired_size = 3
    min_size = 2
    max_size = 6
  }
}

```

---

## 21.13 - Secrets Manager and SSM Parameter Store

secrets\_ssm.tf

```
resource "aws_secretsmanager_secret" "db_creds" {
  name = "efusion/${var.environment}/db-credentials"
}

resource "aws_secretsmanager_secret_version" "db_creds" {
  secret_id = aws_secretsmanager_secret.db_creds.id
  secret_string = jsonencode({ username = "app_user", password =
  random_password.db.result })
}

resource "aws_ssm_parameter" "app_config" {
  name = "/efusion/${var.environment}/app/log-level"
  type = "String"
  value = "INFO"
}
```

**NOTE:** Every resource above uses the `local.common_tags` / `local.name_prefix` pattern from Chapter 8, so a real production file would `merge(local.common_tags, {...})` into every tags argument. It's trimmed here for readability.

---

## Chapter Summary

- A complete AWS environment is built from a small, repeatable set of resource patterns: network, compute, storage, database, load balancing, and observability.
- Security groups should follow least-privilege chaining (ALB -> app tier, not 0.0.0.0/0 -> app tier).
- EKS, ECR, and Lambda extend the same Terraform patterns to container and serverless workloads.

## Best Practices

- Always enable encryption (KMS/SSE) on S3, RDS, and EBS by default - never as an afterthought.
- Use security group chaining (referencing another SG as the source) instead of broad CIDR ranges internally.
- Put `prevent_destroy` on production RDS instances and state-critical S3 buckets.

## Common Mistakes

- Opening security groups to 0.0.0.0/0 for internal-only traffic (app tier, database tier).
- Forgetting `skip_final_snapshot = false` on production RDS, risking silent data loss on destroy.
- Not using `create_before_destroy` on launch templates, causing ASG replacement downtime.

---

## Interview Questions

### Q1. How would you design security groups for a 3-tier web application?

Chain them by reference rather than CIDR: the ALB security group allows inbound 443 from the internet; the app tier security group only allows inbound from the ALB's security group; the database security group only allows inbound from the app tier's security group.

### Q2. What Terraform settings protect a production RDS instance from accidental data loss?

lifecycle { prevent\_destroy = true }, skip\_final\_snapshot = false with a final\_snapshot\_identifier, storage\_encrypted = true, and enabling automated backups with an adequate backup\_retention\_period.

### Q3. How do you deploy a container image built in CI to an EKS cluster using Terraform?

Terraform provisions the EKS cluster, node groups, and ECR repository; the actual image push and Kubernetes deployment rollout is typically handled by CI/CD (or a GitOps tool) after the image is pushed to ECR, since Terraform is best suited to infrastructure, not per-deploy application rollout.

---

# Chapter 22: End-to-End Production Project

This chapter ties every previous chapter together into one realistic, production-grade project layout: a 3-tier web application on AWS, split into independently deployable stacks.

## Folder Structure

Fig 22.1 - Production Folder Structure

```
efusion-infra/
|-- modules/
| |-- vpc/
| |-- alb/
| |-- asg/
| |-- rds/
| \-- eks/
|-- environments/
| |-- dev/
| | |-- main.tf
| | |-- backend.tf
| | \-- dev.tfvars
| |-- qa/
| \-- production/
| |-- main.tf
| |-- backend.tf
| \-- production.tfvars
\-- README.md
```

## environments/production/main.tf

main.tf

```

module "vpc" {
  source = "../../modules/vpc"
  cidr_block = "10.0.0.0/16"
  azs = ["ap-south-1a", "ap-south-1b"]
  name_prefix = "efusion-${var.environment}"
}

module "alb" {
  source = "../../modules/alb"
  vpc_id = module.vpc.vpc_id
  public_subnets = module.vpc.public_subnet_ids
  certificate_arn = var.certificate_arn
}

module "asg" {
  source = "../../modules/asg"
  vpc_id = module.vpc.vpc_id
  private_subnets = module.vpc.private_subnet_ids
  target_group_arn = module.alb.target_group_arn
  desired_capacity = var.environment == "production" ? 4 : 2
}

module "rds" {
  source = "../../modules/rds"
  vpc_id = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnet_ids
  multi_az = var.environment == "production"
}

```

## environments/production/backend.tf

```

terraform {
  required_version = ">= 1.9.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket = "efusion-terraform-state-prod"
    key = "production/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt = true
  }
}

provider "aws" {
  region = "ap-south-1"
}

```

## Deployment and Teardown

```
cd environments/production
terraform init
terraform plan -var-file=production.tfvars -out=tfplan
terraform apply tfplan

# Full environment teardown (non-prod only!)
cd environments/dev
terraform destroy -var-file=dev.tfvars
```

**TIP:** Note the folder-per-environment pattern here: each environment has its own backend.tf pointing to a distinct state key, so a mistake in dev physically cannot touch production's state file.

## Chapter Summary

- A production project separates reusable modules from environment-specific root configurations.
- Each environment gets its own backend key (and ideally its own AWS account/IAM boundary).
- The same module code is reused across dev/qa/production, with only variables changing.

## Best Practices

- Never share a single state file across multiple environments.
- Keep environment root configs thin - they should mostly just call modules with different inputs.
- Store production tfvars (minus secrets) in Git for auditability; keep secrets in Secrets Manager.

## Common Mistakes

- Copy-pasting entire resource blocks between environment folders instead of using shared modules.
- Applying from the wrong environment directory by mistake (always double-check pwd before apply).

## Interview Questions

### Q1. How would you structure a Terraform repository for dev, qa, and production environments?

Reusable modules in a modules/ directory, and thin environment-specific root configurations under environments/dev, environments/qa, environments/production, each with its own backend.tf pointing to a distinct state key (and ideally distinct AWS account), calling the shared modules with different variable values.

### Q2. What's the benefit of keeping environment folders 'thin'?

It minimizes duplicated resource logic - if there's a bug or improvement in how a VPC is built, you fix it once in the module and every environment inherits the fix on their next apply.

---

# Chapter 23: CI/CD Integration

Running terraform apply from a laptop doesn't scale and isn't auditable. Every serious team runs Terraform through a CI/CD pipeline with mandatory plan review before apply.

## GitHub Actions

`.github/workflows/terraform.yml`

```
name: terraform
on:
  pull_request:
  paths: ["environments/production/**"]
  push:
  branches: [main]

jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
    working-directory: environments/production
      - run: terraform plan -var-file=production.tfvars -out=tfplan
    working-directory: environments/production

  apply:
    needs: plan
    if: github.ref == 'refs/heads/main'
    environment: production # requires manual approval gate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
    working-directory: environments/production
      - run: terraform apply -auto-approve tfplan
    working-directory: environments/production
```

## Jenkins Declarative Pipeline

`Jenkinsfile`

```

pipeline {
  agent any
  parameters {
    choice(name: 'ENV', choices: ['dev', 'qa', 'production'], description: 'Target environment')
  }
  stages {
    stage('Init') { steps { sh "terraform -chdir=environments/${params.ENV} init" } }
    stage('Validate') { steps { sh "terraform -chdir=environments/${params.ENV} validate" } }
    stage('Plan') { steps { sh "terraform -chdir=environments/${params.ENV} plan -var-file=${params.ENV}.tfvars -out=tfplan" } }
    stage('Approval') {
      when { expression { params.ENV == 'production' } }
      steps { input message: "Apply to PRODUCTION?" }
    }
    stage('Apply') { steps { sh "terraform -chdir=environments/${params.ENV} apply -auto-approve tfplan" } }
  }
}

```

## GitLab CI

`.gitlab-ci.yml`

```

stages: [validate, plan, apply]

validate:
  stage: validate
  script: terraform -chdir=environments/production validate

plan:
  stage: plan
  script: terraform -chdir=environments/production plan -out=tfplan
  artifacts: { paths: [environments/production/tfplan] }

apply:
  stage: apply
  script: terraform -chdir=environments/production apply -auto-approve tfplan
  when: manual
  only: [main]

```

## Azure DevOps and Bitbucket Pipelines (Quick Reference)

```
# Azure DevOps (azure-pipelines.yml) - key task
- task: TerraformTaskV4@4
inputs:
provider: 'aws'
command: 'apply'
commandOptions: '-auto-approve'
workingDirectory: 'environments/production'

# Bitbucket Pipelines (bitbucket-pipelines.yml)
pipelines:
branches:
main:
- step:
name: Terraform Apply
script:
- cd environments/production
- terraform init
- terraform apply -auto-approve
```

## AWS CodePipeline (native AWS option)

- Source stage: CodeCommit or GitHub connection triggers on push to main.
- Build stage: CodeBuild runs terraform init/plan, uploads the plan as an artifact.
- Manual approval stage: an SNS-notified approval action before production apply.
- Deploy stage: a second CodeBuild project runs terraform apply on the approved plan.

**WARNING:** The single most important CI/CD rule for Terraform: apply must always run against the exact plan file that was reviewed and approved (terraform apply tfplan), never a freshly regenerated plan - otherwise you lose the guarantee that what was approved is what actually runs.

## Chapter Summary

- CI/CD turns Terraform from a manual, unaudited action into a reviewed, repeatable pipeline stage.
- Every major CI/CD tool (GitHub Actions, Jenkins, GitLab, Azure DevOps, Bitbucket, CodePipeline) supports the same init -> plan -> approve -> apply pattern.
- A manual approval gate before production apply is standard practice across all these tools.

## Best Practices

- Always apply the exact saved plan file that was reviewed, not a freshly-generated plan.
- Require a manual approval gate for any pipeline stage targeting production.
- Store pipeline credentials via OIDC federation to the cloud provider, not long-lived static keys.

## Common Mistakes

- 
- Letting -auto-approve run against production without any human approval step.
  - Re-running terraform plan right before apply instead of applying the already-reviewed plan file.
  - Storing AWS access keys as plaintext CI/CD variables instead of using OIDC/IAM role federation.

## Interview Questions

### **Q1. Why should terraform apply run against a saved plan file rather than a fresh plan in a pipeline?**

Because between the reviewed plan and apply time, underlying state or resources could drift, silently changing what actually gets applied. Using terraform apply guarantees the exact reviewed changes are what execute.

### **Q2. How do you enforce human approval before a production Terraform apply in CI/CD?**

Using a manual approval gate - a GitHub Actions 'environment' protection rule, a Jenkins input step, a GitLab 'when: manual' job, or a CodePipeline manual approval action - placed between the plan and apply stages.

### **Q3. Why is OIDC federation preferred over static access keys in a CI/CD pipeline?**

OIDC lets the CI/CD platform assume a short-lived, scoped IAM role for the duration of the job, eliminating the need to store and rotate long-lived static AWS credentials as pipeline secrets.

---

# Chapter 24: Terraform Best Practices

## Naming Standards

```
# Pattern: <project>-<environment>-<resource-type>-<optional-qualifier>
efusion-prod-vpc
efusion-prod-alb-public
efusion-prod-rds-primary
efusion-dev-eks-cluster
```

## Folder Structure Standard

- modules/ for reusable, versioned building blocks.
- environments// for thin root configurations calling modules.
- One backend key per environment/stack, never shared.

## Secrets Management

- Never hardcode secrets in .tf or .tfvars files committed to Git.
- Use AWS Secrets Manager or SSM Parameter Store (SecureString) and read via data sources.
- Mark all secret-carrying variables sensitive = true, and encrypt the state backend.

## Least Privilege

- CI/CD pipeline IAM roles should have only the permissions needed for that specific stack.
- Never grant AdministratorAccess to a Terraform execution role, even 'temporarily'.
- Use separate IAM roles per environment so a dev pipeline mistake cannot touch production.

## Versioning

- Pin Terraform core version with required\_version in the terraform {} block.
- Pin provider versions with ~>.
- Pin module sources to a Git tag or registry version, never a floating branch.

## State Security

- Enable versioning and SSE-KMS encryption on the state bucket.
- Restrict IAM access to the state bucket/table to the smallest necessary group.
- Use DynamoDB locking (or an equivalent) to prevent concurrent-apply corruption.

## Performance Optimization

- 
- Split large monolithic configurations into smaller stacks (network, compute, database) to keep plan/apply times manageable.
  - Use -target sparingly for emergency fixes only, not as a regular workflow.
  - Use data source filters tightly to avoid unnecessary API calls on every plan.
  - Run terraform plan with -parallelism tuned for very large configurations if provider API rate limits are hit.

**TIP:** A useful rule of thumb: if terraform plan takes more than 2-3 minutes routinely, or a single state file has more than a few hundred resources, it's time to split the stack.

---

## Chapter Summary

- Consistent naming, thin environment folders, and versioned modules are the foundation of maintainable Terraform.
- Secrets belong in Secrets Manager/SSM, never in plaintext code or tfvars.
- Least privilege IAM, pinned versions, and encrypted state backends are non-negotiable for production.

## Best Practices

- Adopt a single naming convention across the entire organization, documented in a README.
- Split large state files into smaller, purpose-specific stacks as they grow.
- Automate terraform fmt and terraform validate as pre-commit hooks.

## Common Mistakes

- Letting one giant state file accumulate hundreds of unrelated resources over time.
- Granting broad IAM permissions to a CI/CD role 'just to get it working', then never tightening it.

## Interview Questions

### Q1. What are the top 5 Terraform best practices you follow in production?

Consistent naming conventions, modular thin-environment structure, encrypted and locked remote state, least-privilege IAM per environment, and pinned versions for Terraform core, providers, and modules.

### Q2. When should you split a Terraform state file into multiple smaller stacks?

When plan/apply times become slow, when the blast radius of a single mistake becomes too large, or when different teams need independent deployment cadences for different parts of the infrastructure (e.g., network team vs application team).

---

# Chapter 25: Production Troubleshooting

## State Lock Issues

```
Error: Error acquiring the state lock
Lock Info:
ID: 8f3a2b1c-...
Path: efusion-terraform-state-prod/production/terraform.tfstate
```

Confirm no other terraform process is actually running (check CI/CD job status), then:

```
terraform force-unlock 8f3a2b1c-...
```

## Provider Errors

- **AuthFailure / ExpiredToken:** refresh AWS credentials (SSO session or assumed role expired).
- **UnsupportedOperation:** often a region/version mismatch - check the resource is available in that region.
- **Throttling: Rate exceeded:** reduce -parallelism or add retry logic in the provider block.

## Dependency Errors

```
Error: Cycle: aws_security_group.a, aws_security_group.b
```

Circular dependencies usually come from security groups referencing each other directly. Fix by using a separate `aws_security_group_rule` resource to break the cycle, or restructure the reference direction.

## Timeouts

```
resource "aws_db_instance" "main" {
  # ...
  timeouts {
    create = "60m"
    update = "80m"
    delete = "40m"
  }
}
```

## Permission Issues

- **AccessDenied** errors almost always mean the Terraform execution role/user is missing an IAM action.
- Use IAM Access Analyzer or CloudTrail to identify exactly which action was denied.
- Add the minimum required action, not a wildcard, to keep least privilege intact.

---

## Drift Detection and Correction

```
terraform apply -refresh-only # detect drift without changing resources
terraform plan # shows the diff between state and real infra
terraform apply # reconcile config back to desired state (or update code to match reality)
```

## Debugging with TF\_LOG

```
export TF_LOG=DEBUG
export TF_LOG_PATH=./terraform-debug.log
terraform apply
```

**TIP:** TF\_LOG=TRACE produces extremely verbose output including raw provider API requests - useful for filing a provider bug report, but almost always too noisy for everyday debugging. Start with TF\_LOG=INFO.

---

## Chapter Summary

- State lock errors are resolved with force-unlock, only after confirming no other process is running.
- Provider errors are usually credentials, region availability, or API throttling.
- Dependency cycles are almost always broken by decoupling security group rules into separate resources.
- TF\_LOG lets you inspect exactly what Terraform is sending to and receiving from the provider API.

## Best Practices

- Always verify no other process holds a lock before force-unlocking.
- Add explicit timeouts blocks for slow-to-provision resources like RDS and EKS.
- Use -refresh-only regularly in production to catch drift before it causes a surprising apply.

## Common Mistakes

- Force-unlocking a state lock while a legitimate apply is still mid-flight, corrupting state.
- Ignoring throttling errors instead of tuning -parallelism or adding retry logic.
- Debugging blind without ever enabling TF\_LOG, wasting hours guessing at the root cause.

## Interview Questions

**Q1. A colleague's terraform apply crashed mid-run and now every apply fails with a lock error. What do you do?**

---

First confirm via CI/CD history or process monitoring that no process is genuinely still running. Then run terraform force-unlock with the lock ID from the error message to manually release the stuck lock, and re-run plan to confirm state integrity.

**Q2. How do you troubleshoot a Terraform provider throwing intermittent throttling errors?**

Reduce the -parallelism value to lower concurrent API calls, check whether the provider block supports built-in retry configuration, and review CloudTrail/service quotas for the account.

**Q3. What causes a Terraform dependency cycle error and how do you fix it?**

Two or more resources reference each other directly (e.g., security groups allowing each other's SG ID inline). Fix by extracting the rule into a standalone aws\_security\_group\_rule resource so the reference direction is one-way instead of circular.

---

## PART V - CAREER & MASTERY

# Chapter 26: Terraform Interview Questions

This chapter is a curated, high-signal interview bank spanning beginner through advanced levels, plus scenario-based questions. Every chapter in this book also ends with its own topic-specific questions - combined, this book carries well over 100 unique interview questions with complete answers.

## Beginner Level

### Q1. What problem does Terraform solve?

It replaces manual, undocumented infrastructure provisioning with version-controlled, repeatable code, tracked against a state file.

### Q2. What language does Terraform use?

HCL (HashiCorp Configuration Language), a declarative, human-readable configuration language.

### Q3. What is a provider in Terraform?

A plugin that lets Terraform talk to a specific API, such as AWS, Azure, GCP, or Kubernetes.

### Q4. What is the difference between terraform plan and terraform apply?

plan shows a dry-run diff of proposed changes; apply executes those changes against real infrastructure.

### Q5. What is the Terraform state file?

A JSON file that maps your configuration to real resource IDs, used to determine what changes are needed on the next plan/apply.

### Q6. What command initializes a Terraform working directory?

terraform init, which downloads providers, configures the backend, and fetches modules.

### Q7. What is a resource block?

The core building block representing a single infrastructure object to be created and managed, e.g. resource "aws\_instance" "web" {...}.

### Q8. What is the purpose of variables in Terraform?

To parameterize configuration so the same code can be reused across environments with different input values.

### Q9. What is an output value used for?

To expose specific resource attributes after apply, for humans, pipelines, or other modules to consume.

### Q10. What does terraform fmt do?

Rewrites .tf files into canonical formatting and indentation for consistency across a team.

### Q11. What is a data source?

A read-only query for information about infrastructure Terraform doesn't manage, such as an existing VPC or the latest AMI.

### Q12. How do you supply a variable value from the command line?

Using the -var flag, e.g. terraform apply -var="instance\_type=t3.large".

---

**Q13. What is the difference between a list and a map in Terraform?**

A list is an ordered collection accessed by index; a map is a set of key-value pairs accessed by key.

**Q14. What file extension do Terraform configuration files use?**

The .tf extension for HCL files, and .tfvars for variable definition files.

**Q15. What does terraform destroy do?**

Destroys every resource tracked in the current state file, in reverse dependency order.

---

## Intermediate Level

### Q1. What is the difference between count and for\_each?

count creates resources indexed numerically, causing reordering issues if items are removed from the middle of a list. for\_each keys resources by a stable identifier from a map or set, so adding/removing one item doesn't disturb the others.

### Q2. How does Terraform determine resource creation order?

By parsing attribute references between resources to build a Directed Acyclic Graph (DAG); independent resources are created in parallel, dependent ones in order.

### Q3. What is a local value and how is it different from a variable?

A local is an internal, computed value private to a module, used to avoid repeating expressions. A variable is an external input settable by the caller.

### Q4. How do you manage secrets in Terraform?

Store them in AWS Secrets Manager or SSM Parameter Store and read them via data sources at plan/apply time; mark related variables sensitive = true and always encrypt the state backend.

### Q5. What is a Terraform module?

A reusable, self-contained directory of .tf files with its own variables and outputs, callable from a root or other module.

### Q6. What is the purpose of the lifecycle block?

It controls special resource behaviors: create\_before\_destroy for zero-downtime replacement, prevent\_destroy to block accidental deletion, and ignore\_changes to suppress drift on specific attributes.

### Q7. What's the difference between implicit and explicit dependencies?

Implicit dependencies are inferred automatically from attribute references between resources. Explicit dependencies are declared via depends\_on when no direct reference exists but ordering still matters.

### Q8. How do you handle multiple environments (dev/qa/prod) in Terraform?

Either via Terraform workspaces sharing one backend, or (more common in regulated environments) via separate root configuration directories with separate backends/state keys and separate IAM boundaries per environment.

### Q9. What does terraform import do, and what are its limitations?

It links an existing, manually-created cloud resource into the state file so Terraform can manage it going forward. It requires a matching resource block to already exist in code, and it does not generate that code automatically in older Terraform versions.

### Q10. How do dynamic blocks work?

They generate repeated nested configuration blocks (like security group ingress rules) from a list or map input, using a for\_each-style iterator exposing .key and .value.

### Q11. What is remote state and why is it necessary for teams?

State stored in a shared, durable, lockable backend (like S3+DynamoDB) instead of a local file, so multiple engineers can safely collaborate without state conflicts or corruption.

### Q12. What's the difference between terraform taint and -replace?

---

taint is a deprecated command that marks a resource for recreation outside the plan. -replace (e.g., terraform apply -replace=aws\_instance.web) is the modern equivalent, showing the replacement clearly inside the plan output before executing.

**Q13. How do you reference the output of one Terraform stack from another?**

Using the terraform\_remote\_state data source configured with the other stack's backend settings, then referencing data.terraform\_remote\_state.X.outputs.Y.

**Q14. What happens if two engineers run terraform apply at the same time without state locking?**

Both processes can read and write to the state file concurrently, corrupting it and potentially causing partial, inconsistent infrastructure changes - this is exactly what backends like S3+DynamoDB locking prevent.

**Q15. What is drift and how do you detect it?**

Drift is when real infrastructure no longer matches the Terraform configuration/state, often from manual console changes. terraform plan surfaces drift as a diff; terraform apply -refresh-only detects it without making changes.

---

## Advanced Level

### Q1. How would you design a multi-account, multi-region Terraform architecture for a bank?

Separate AWS accounts per environment (dev/qa/production) with account-level IAM boundaries, separate state backends per account/stack, provider aliasing for cross-region resources (like ACM certs for CloudFront), and a centralized module registry versioned via Git tags, deployed through a CI/CD pipeline with mandatory approval gates for production.

### Q2. How do you safely refactor a resource into a module without downtime?

Use terraform state mv to move the resource's state address into the new module path, matching the exact same resource configuration inside the module, then run terraform plan to confirm zero diff before committing the refactor.

### Q3. Explain how Terraform's dependency graph enables parallel resource creation.

Terraform builds a DAG from all references between resources. Any resources that don't depend on each other, directly or transitively, are created concurrently up to the configured -parallelism limit, which is why Terraform apply is often much faster than sequential shell-script provisioning.

### Q4. How would you enforce policy-as-code (e.g., 'no public S3 buckets') across all Terraform runs?

Integrate a policy engine like HashiCorp Sentinel (in HCP Terraform) or Open Policy Agent (OPA)/Conftest in the CI/CD pipeline, running policy checks against the plan's JSON output before allowing apply.

### Q5. What's the difference between a Terraform provider and a Terraform provisioner?

A provider is a plugin translating HCL resource/data blocks into API calls for a specific service. A provisioner runs arbitrary local or remote commands as part of resource creation/destruction, and is considered a last-resort tool compared to providers, modules, and dedicated configuration management.

### Q6. How do you handle a scenario where a shared module change needs to roll out gradually across environments?

Version the module with Git tags; bump the ref in the dev environment first, validate with plan/apply, then progressively bump qa and finally production, treating the module version bump itself as a reviewed code change per environment.

### Q7. How would you migrate from a monolithic single state file to multiple smaller stacks with zero resource recreation?

Use terraform state mv (or the newer moved blocks) to relocate each resource's state address into its new stack's state file without destroying and recreating, carefully validating with plan after each move that no diff is introduced.

### Q8. What are the tradeoffs of using count vs for\_each when a list of instances might be resized?

count reindexes all subsequent resources when an item is removed from the middle, causing unnecessary destroy/recreate operations on unrelated resources. for\_each avoids this entirely by keying off a stable identifier, making it the safer default for anything beyond a fixed, ordinal count.

### Q9. How do you handle Terraform provider rate limiting at scale (hundreds of resources)?

Reduce -parallelism, split the configuration into smaller independently-applied stacks, and where the provider supports it, configure built-in retry/backoff settings in the provider block.

### Q10. Describe how you would build a zero-downtime blue-green deployment using Terraform for an ASG-backed application.

---

Use `create_before_destroy` on the launch template, provision a new target group and ASG in parallel with the old one, shift ALB listener rules or weighted target groups to the new ASG once healthy, then destroy the old ASG and target group in a follow-up apply.

---

## Scenario-Based Questions

**Q1. A production apply is stuck mid-run and the pipeline job was killed. The next run fails with a state lock error. What do you do?**

Verify in the CI/CD system and cloud console that no process is genuinely still executing (no partial resources being created), then run terraform force-unlock using the ID from the error, and re-run terraform plan to confirm the state accurately reflects reality before proceeding.

**Q2. Someone manually deleted an S3 bucket that Terraform manages, outside of Terraform. What happens on the next apply?**

Terraform's state still references the bucket. Depending on the resource and Terraform version, the next plan will typically detect the missing resource and propose to recreate it; if recreation isn't desired (e.g., the bucket is being decommissioned intentionally), run terraform state rm to stop tracking it instead.

**Q3. Your team needs to give a new junior engineer access to run terraform plan but not apply in production. How do you set this up?**

Use IAM permission boundaries or separate CI/CD pipeline roles - a read-only/plan role with only Describe/Get/List permissions for plan, and a separate, more privileged apply role gated behind a manual approval step that only senior engineers can trigger.

**Q4. A module you maintain is used by 12 downstream teams. You need to add a new required field to one of its resources. How do you avoid breaking all 12 teams at once?**

Add the field with a sensible default value so existing callers are unaffected, release it as a new minor version tag, and only make it a hard requirement (breaking change) in a major version bump communicated and rolled out on each team's own schedule.

**Q5. A colleague wants to use terraform apply -auto-approve directly against production from their laptop 'just this once' to fix an urgent incident. What do you tell them?**

Explain the risk: no plan review, no audit trail through the pipeline, and no lock coordination with the shared backend if others are also working. Recommend running it through the existing pipeline with an expedited approval instead, or if truly emergency and unavoidable, ensure they coordinate with the team, confirm no one else is applying, and document the change immediately afterward.

---

## Chapter Summary

- This chapter plus every prior chapter's own Q&A; section together form a 100+ question interview bank.
- Beginner questions test vocabulary and command recall; intermediate tests judgment on tradeoffs; advanced tests architecture and failure-mode reasoning.
- Scenario questions simulate real incidents - practice narrating your reasoning out loud, not just the final answer.

---

# Chapter 27: Real Production Scenarios

## 27.1 - Infrastructure Migration (Lift and Shift into Terraform)

Scenario: you inherit a production AWS account built entirely by hand. Approach: inventory every resource (AWS Config or a tool like former2 helps generate a starting point), write matching resource blocks, import each one with terraform import, and iterate until terraform plan shows zero diff before making any code changes.

## 27.2 - Blue-Green Deployment

`blue_green.tf`

```
# Two parallel target groups behind one ALB listener, weighted
resource "aws_lb_listener_rule" "blue_green" {
  listener_arn = aws_lb_listener.https.arn
  action {
    type = "forward"
    forward {
      target_group { arn = aws_lb_target_group.blue.arn; weight = var.blue_weight }
      target_group { arn = aws_lb_target_group.green.arn; weight = var.green_weight }
    }
  }
}
```

Shift `var.blue_weight/green_weight` gradually (100/0 -> 50/50 -> 0/100) across successive applies, monitoring error rates at each step, then decommission the old target group once green is fully validated.

## 27.3 - Disaster Recovery (Multi-Region Standby)

- Primary stack deployed in `ap-south-1`; a near-identical DR stack in `ap-southeast-1`, both from the same modules.
- RDS cross-region read replica or automated snapshot copy for database DR.
- Route53 failover routing policy with health checks pointed at the primary ALB.
- Terraform apply for the DR stack is tested regularly (not just written once and forgotten) via a scheduled non-production drill.

## 27.4 - Multi-Region and Multi-Account Architecture

```
provider "aws" {
  alias = "primary"
  region = "ap-south-1"
}
provider "aws" {
  alias = "dr"
  region = "ap-southeast-1"
}

module "primary_stack" {
  source = "../modules/full-stack"
  providers = { aws = aws.primary }
}

module "dr_stack" {
  source = "../modules/full-stack"
  providers = { aws = aws.dr }
}
```

## 27.5 - Zero Downtime Deployment

- `create_before_destroy` on launch templates so the new version boots and passes health checks before old instances terminate.
- ASG instance refresh with a `min_healthy_percentage` above 50% to roll gradually.
- ALB connection draining (`deregistration_delay`) to let in-flight requests finish before instance termination.

## 27.6 - Scaling Strategy

- Target tracking scaling policies (CPU, request count per target) for predictable, self-adjusting capacity.
- Scheduled scaling actions for known traffic patterns (e.g., scale up before end batch processing in BFSI).
- Split large ASGs across multiple AZs and use a database read-replica tier for read-heavy scaling separate from write scaling.

---

## Chapter Summary

- Real production scenarios combine multiple Terraform features: modules, provider aliasing, lifecycle rules, and weighted routing.
- Migration into Terraform is done via careful, iterative import - never a single big-bang rewrite.
- Disaster recovery infrastructure must be tested regularly, not just written and left untouched.

## Best Practices

- Test your DR failover procedure on a schedule, not only during an actual incident.
- Always validate blue-green weight shifts gradually with real monitoring at each step, not a single instant cutover.

---

## Common Mistakes

- Writing DR infrastructure once and never actually testing a failover until a real outage forces it.
- Cutting over 100% traffic in a blue-green deploy without a gradual, monitored weight shift.

## Interview Questions

### Q1. How would you migrate a hand-built AWS account to Terraform without downtime?

Inventory existing resources, write matching resource blocks, import each with terraform import, and iterate until plan shows zero diff before making any further code changes - never destroy and recreate existing production resources during migration.

### Q2. How do you implement a blue-green deployment using an ALB in Terraform?

Maintain two target groups (blue and green) behind one ALB listener rule using weighted forwarding, gradually shifting the weight from the old target group to the new one while monitoring health and error rates at each step.

---

# Chapter 28: Terraform Cheat Sheets

## Commands

| Command                       | Purpose                            |
|-------------------------------|------------------------------------|
| terraform init                | Initialize working directory       |
| terraform validate            | Check syntax                       |
| terraform plan                | Preview changes                    |
| terraform apply               | Execute changes                    |
| terraform destroy             | Tear down all resources            |
| terraform fmt -recursive      | Format all files                   |
| terraform show                | Human-readable state/plan          |
| terraform output              | Print output values                |
| terraform import              | Bring existing resource into state |
| terraform apply -replace=     | Force resource recreation          |
| terraform state list          | List resources in state            |
| terraform workspace list      | List workspaces                    |
| terraform graph               | Dependency graph (DOT format)      |
| terraform apply -refresh-only | Detect drift, no changes           |

## Functions Quick Reference

| Function                  | Example                              |
|---------------------------|--------------------------------------|
| merge()                   | merge(local.common_tags, {Name="x"}) |
| length()                  | length(var.subnet_ids)               |
| lookup()                  | lookup(var.map, "key", "default")    |
| jsonencode()/jsondecode() | jsonencode({a=1})                    |
| templatefile()            | templatefile("tpl.tftpl", {n="x"})   |
| cidrsubnet()              | cidrsubnet("10.0.0.0/16", 8, 1)      |
| toset()/tomap()           | toset(["a","b"])                     |

| Function       | Example                        |
|----------------|--------------------------------|
| for expression | [for x in var.list : upper(x)] |

## Meta-Arguments & Lifecycle

| Meta-Argument                   | Use                                                  |
|---------------------------------|------------------------------------------------------|
| count                           | Numeric repetition                                   |
| for_each                        | Keyed repetition (preferred for named resources)     |
| depends_on                      | Explicit ordering when no attribute reference exists |
| provider                        | Select a specific (aliased) provider instance        |
| lifecycle.create_before_destroy | Zero-downtime replacement                            |
| lifecycle.prevent_destroy       | Block accidental destroy                             |
| lifecycle.ignore_changes        | Suppress drift on specific attributes                |

## Variables, Outputs, State - At a Glance

```
# Variable
variable "x" { type = string; default = "v"; sensitive = false }

# Output
output "x" { value = aws_vpc.main.id; sensitive = false }

# Backend
terraform {
  backend "s3" {
    bucket = "..."; key = "..."; region = "..."; dynamodb_table = "..."; encrypt = true
  }
}
```

## Chapter Summary

- This chapter is a fast-lookup reference for commands, functions, meta-arguments, and backend syntax.
- Bookmark this chapter for daily use - it condenses the entire book into one page you can scan in seconds.

## Best Practices

- Print or pin this chapter near your desk during your first few months of daily Terraform work.

---

## Common Mistakes

- Relying on memory for exact function argument order instead of checking this reference under deadline pressure.

## Interview Questions

### Q1. Name three Terraform meta-arguments and what each controls.

count and for\_each control resource repetition (numeric vs keyed); depends\_on controls explicit ordering; lifecycle controls replacement/destroy/drift behavior.

---

# Chapter 29: Hands-On Labs

20 progressive labs, from your first terraform init to a full production-style deployment. Each lab lists the goal, the steps, and how to validate you got it right.

## Lab 1: Install Terraform and verify the version

**Steps:** Install via your OS package manager; run terraform -version.

**Validation:** Command prints a version  $\geq$  1.9.0.

## Lab 2: Create your first resource

**Steps:** Write a provider block and a single aws\_s3\_bucket resource; run init, plan, apply.

**Validation:** Bucket appears in the AWS Console; terraform state list shows it.

## Lab 3: Use variables and outputs

**Steps:** Parameterize the bucket name with a variable; output the bucket ARN.

**Validation:** terraform output shows the correct ARN.

## Lab 4: Build a VPC with public/private subnets

**Steps:** Use the code from Chapter 21.1.

**Validation:** Two subnets visible in VPC Console, correctly tagged.

## Lab 5: Add an Internet Gateway and NAT Gateway

**Steps:** Extend Lab 4 with IGW/NAT and route tables.

**Validation:** Public subnet reaches internet; private subnet reaches internet only via NAT.

## Lab 6: Launch an EC2 instance into a private subnet

**Steps:** Use a security group referencing the ALB's SG as source.

**Validation:** Instance has no public IP but is reachable from the ALB tier only.

## Lab 7: Create an Application Load Balancer

**Steps:** Wire ALB, target group, and listener from Chapter 21.7.

**Validation:** ALB DNS name returns a healthy response after registering the instance.

## Lab 8: Convert to a reusable module

**Steps:** Move the VPC code into modules/vpc with variables/outputs.

**Validation:** Root config calls module "vpc" with zero diff after refactor (use state mv).

---

## Lab 9: Configure a remote S3 + DynamoDB backend

**Steps:** Bootstrap bucket and lock table, then migrate state with terraform init -migrate-state.

**Validation:** terraform state list works against the new S3-backed state.

## Lab 10: Use for\_each to create 3 named EC2 instances

**Steps:** Replace count with for\_each keyed by service name.

**Validation:** Instance tags show api/worker/scheduler, not 0/1/2.

## Lab 11: Add lifecycle protection to an RDS instance

**Steps:** Add prevent\_destroy = true and confirm destroy fails.

**Validation:** terraform destroy errors with a lifecycle protection message.

## Lab 12: Write a dynamic block for security group rules

**Steps:** Parameterize ingress rules via a list variable.

**Validation:** Adding a new rule to the variable creates a new ingress block with no code changes.

## Lab 13: Import an existing manually-created S3 bucket

**Steps:** Create it manually first, then import it.

**Validation:** terraform plan shows zero diff after import.

## Lab 14: Practice state surgery

**Steps:** Use state mv to rename a resource without destroying it.

**Validation:** Resource ID stays identical in AWS; only the Terraform address changes.

## Lab 15: Set up two workspaces (dev/staging)

**Steps:** Create workspaces and vary instance\_type by terraform.workspace.

**Validation:** Each workspace has an independent state file and instance size.

## Lab 16: Build a CI/CD pipeline with GitHub Actions

**Steps:** Use the workflow from Chapter 23.

**Validation:** PR triggers plan as a check; merge to main triggers apply after approval.

## Lab 17: Add CloudWatch alarms and SNS notifications

**Steps:** Wire an alarm to notify on high CPU.

**Validation:** Manually trigger load and confirm an SNS email/notification fires.

---

## Lab 18: Deploy an EKS cluster with a node group

**Steps:** Use Chapter 21.12 code.

**Validation:** `kubectl get nodes` shows the expected number of Ready nodes.

## Lab 19: Simulate and recover from a state lock

**Steps:** Kill an apply mid-run, then force-unlock safely.

**Validation:** Subsequent plan/apply succeeds without state corruption.

## Lab 20: Full production project

**Steps:** Assemble Chapters 21-24 into one end-to-end environment.

**Validation:** A public URL serves traffic through ALB -> ASG -> app, with RDS behind it, entirely provisioned by terraform apply.

---

## Chapter Summary

- These 20 labs progress from a single resource to a full production-grade environment.
- Each lab has an explicit validation step - don't move on until you've verified the result yourself.

---

# Chapter 30: Final Notes - Career Roadmap

## Learning Path

- **Weeks 1-2:** Chapters 1-10 - install Terraform, master HCL basics, variables, outputs, resources.
- **Weeks 3-4:** Chapters 11-15 - data sources, expressions, functions, meta-arguments, dynamic blocks.
- **Weeks 5-6:** Chapters 16-20 - modules, remote state, state internals, workspaces, provisioners.
- **Weeks 7-9:** Chapters 21-23 - build the full AWS production project and wire it into CI/CD.
- **Weeks 10-12:** Chapters 24-29 - best practices, troubleshooting, interview prep, and all 20 labs.

## Certification Guide

| Certification                                  | Level         | Notes                                                                    |
|------------------------------------------------|---------------|--------------------------------------------------------------------------|
| HashiCorp Certified: Terraform Associate (003) | Foundational  | Covers exactly the scope of Chapters 1-20; strong first credential.      |
| AWS Certified Solutions Architect - Associate  | Complementary | Deepens the AWS knowledge behind Chapter 21's resources.                 |
| AWS Certified DevOps Engineer - Professional   | Advanced      | Pairs well with Chapters 22-25 (CI/CD, troubleshooting, production ops). |

## Useful Resources

- Official docs: [developer.hashicorp.com/terraform](https://developer.hashicorp.com/terraform)
- Terraform Registry (modules/providers): [registry.terraform.io](https://registry.terraform.io)
- AWS provider changelog: [github.com/hashicorp/terraform-provider-aws/releases](https://github.com/hashicorp/terraform-provider-aws/releases)
- Community: HashiCorp Discuss forums, [r/Terraform](https://www.reddit.com/r/Terraform)

## Final Summary

You've gone from 'what is Infrastructure as Code' to designing multi-region, multi-account production architectures with CI/CD guardrails. The gap between knowing Terraform syntax and being production-ready is entirely closed by repetition: rebuild the Chapter 22 project from memory, break it on purpose, and fix it using Chapter 25. That cycle, repeated a handful of times, is what actually builds interview-ready confidence.

**Thank you for reading.**

[infracraft.dev](https://infracraft.dev) | [@engineer\\_cloudops](https://twitter.com/engineer_cloudops)